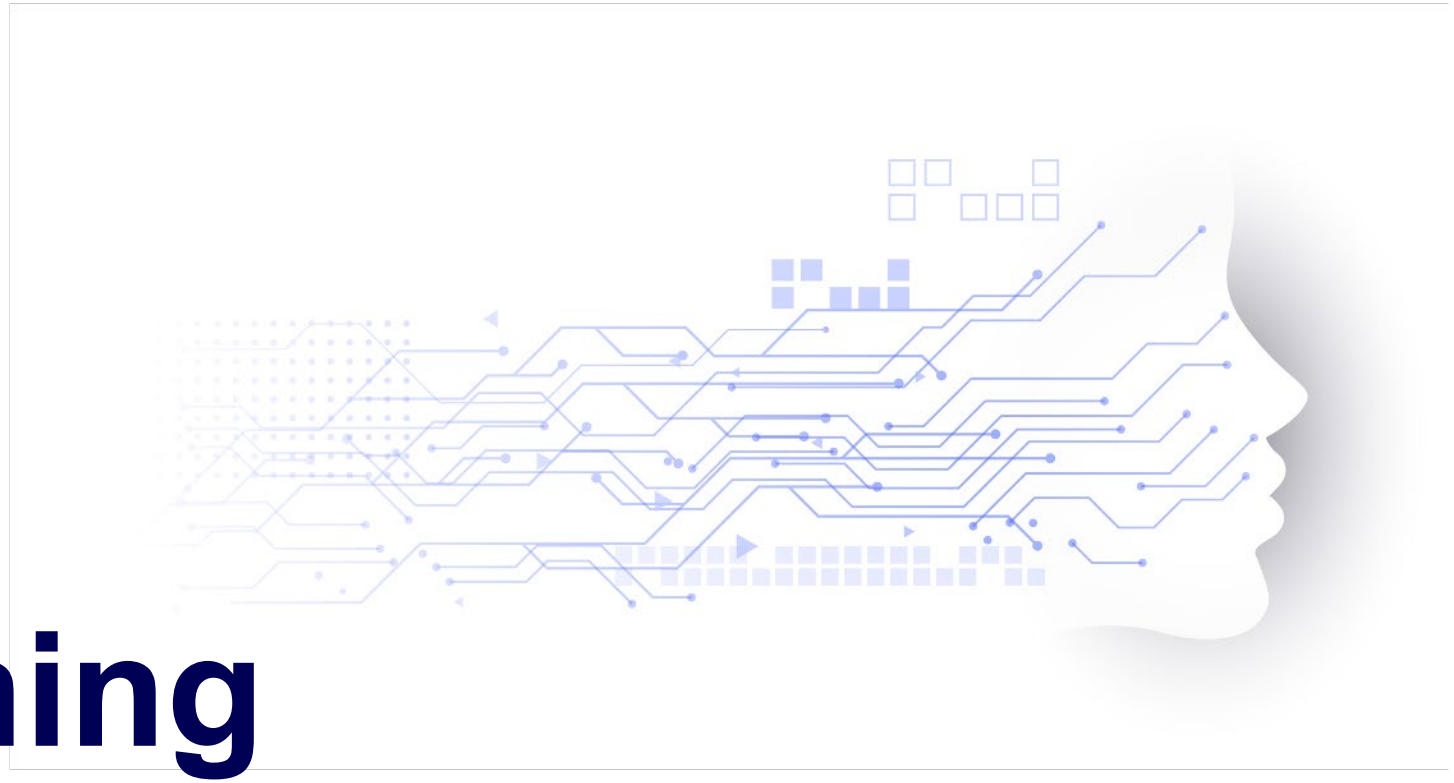
RMIT Classification: Trusted



Deep Learning

COSC 2673/2793

(Computational) Machine Learning

Image: Freepik.com

RMIT UNIVERSITY

Agenda

- Video Lectures:
  - Introduction to Deep learning
  - Computer vision basics
  - MLP to CNN
  - CNN Architectures
  - Practical Aspects

Assignment 2

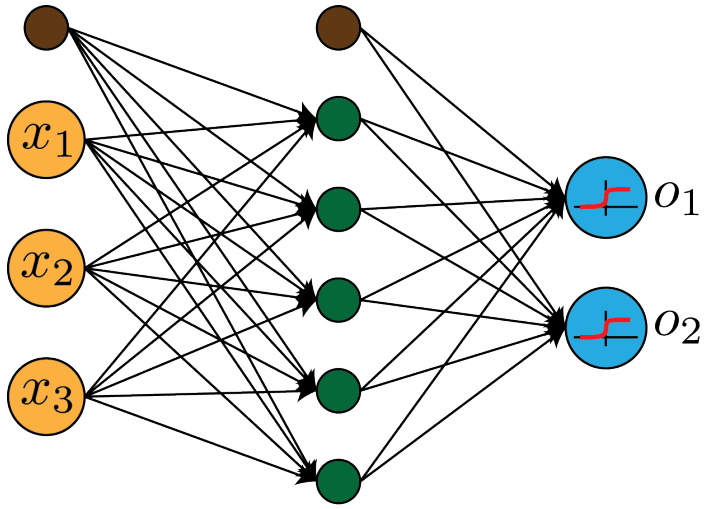
Q&A

Things to consider for Assignment 2

- Did you pick the correct techniques (for all elements of the pipeline)? Do you have evidence to show why you selected these techniques?
- Have you identified all the parameters, and did you set them correctly?
- Did you carry out the evaluations fairly and consistently?
  - Is your evaluation method robust?
  - Did you pick the correct metrics? Why?
- Evidence based ultimate judgment (Not just best MSE -> Best model).

The above is applicable when solving any ML problem

Multi-Layer Perceptron (MLP)



$$h_{\mathbf{w}}\left(\mathbf{x}^{(i)}\right) = \hat{y}^{(i)} = \sigma\left(\mathbf{W}^{(2)}\sigma\left(\mathbf{W}^{(1)}\mathbf{x}^{(i)}\right)\right)$$

$$h_{\mathbf{w}}\left(\mathbf{X}^{(i)}\right) = \hat{y}^{(i)} = \left[o_1^{(i)}, o_2^{(i)}, \cdots, o_c^{(i)}\right]$$

Experience: $D = \left\{\left(\mathbf{X}^{(i)}, y^{(i)}\right)\right\}_{i=1}^{n}$

Hypothesis space: Non- linear when activation is non-linear.

Loss: Log-loss (binary/categorical cross-entropy)

$$\mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right) = -y^{(i)}\log\left(\hat{y}^{(i)}\right) - \left(1 - y^{(i)}\right)\log\left(1 - \hat{y}^{(i)}\right)$$

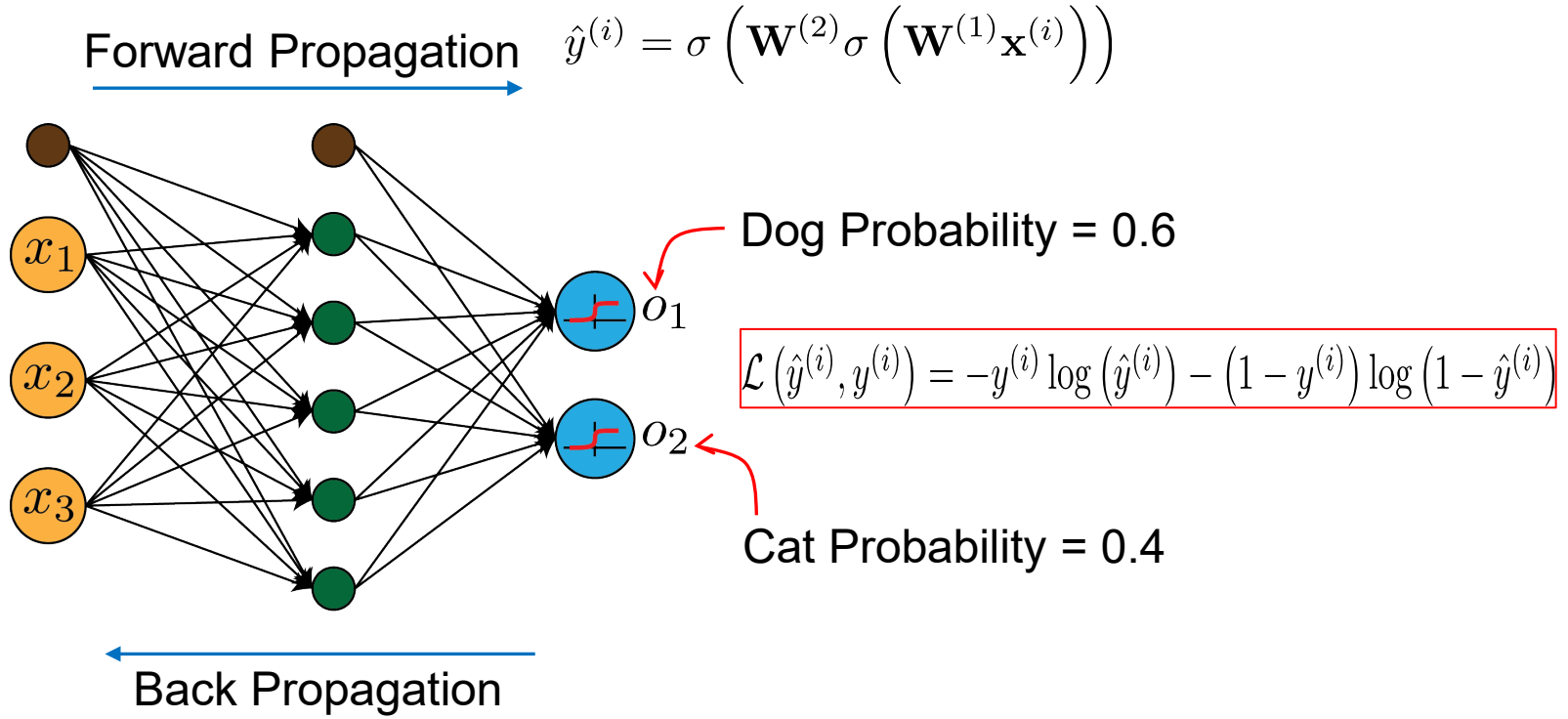
$$\mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right) = -\sum_{j=0}^{c} y_j^{(i)}\log\left(\hat{y}_j^{(i)}\right) = -\sum_{j=0}^{c} y_j^{(i)}\log\left(o_j^{(i)}\right)$$

Optimization: Back propagation (stochastic gradient decent)

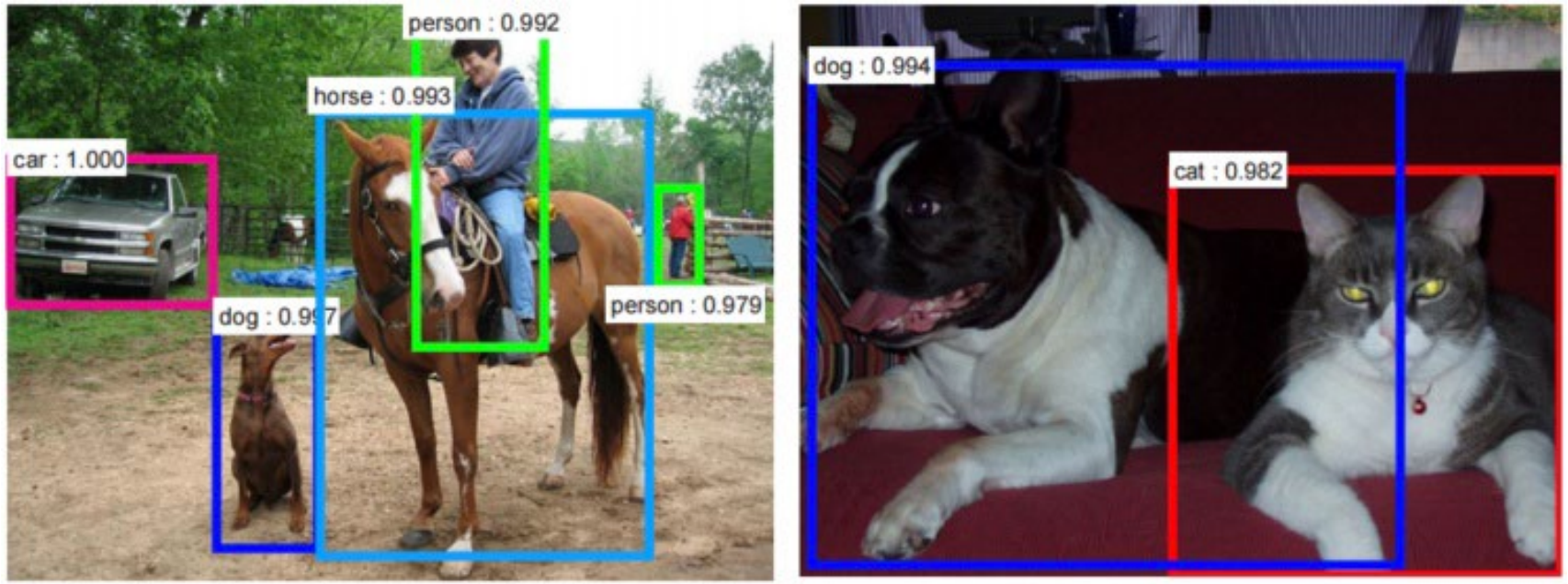
This approach gives up on the convexity of the training problem, but the benefits outweigh the harms. – Ian Good fellow, Deep learning book.

Multi-Layer Perceptron (MLP)





Object Detection & Recognition



Source: https://arxiv.org/pdf/1506.01497v3.pdf

Object Detection & Recognition



a soccer player is kicking a soccer ball



a street sign on a pole in front of a building



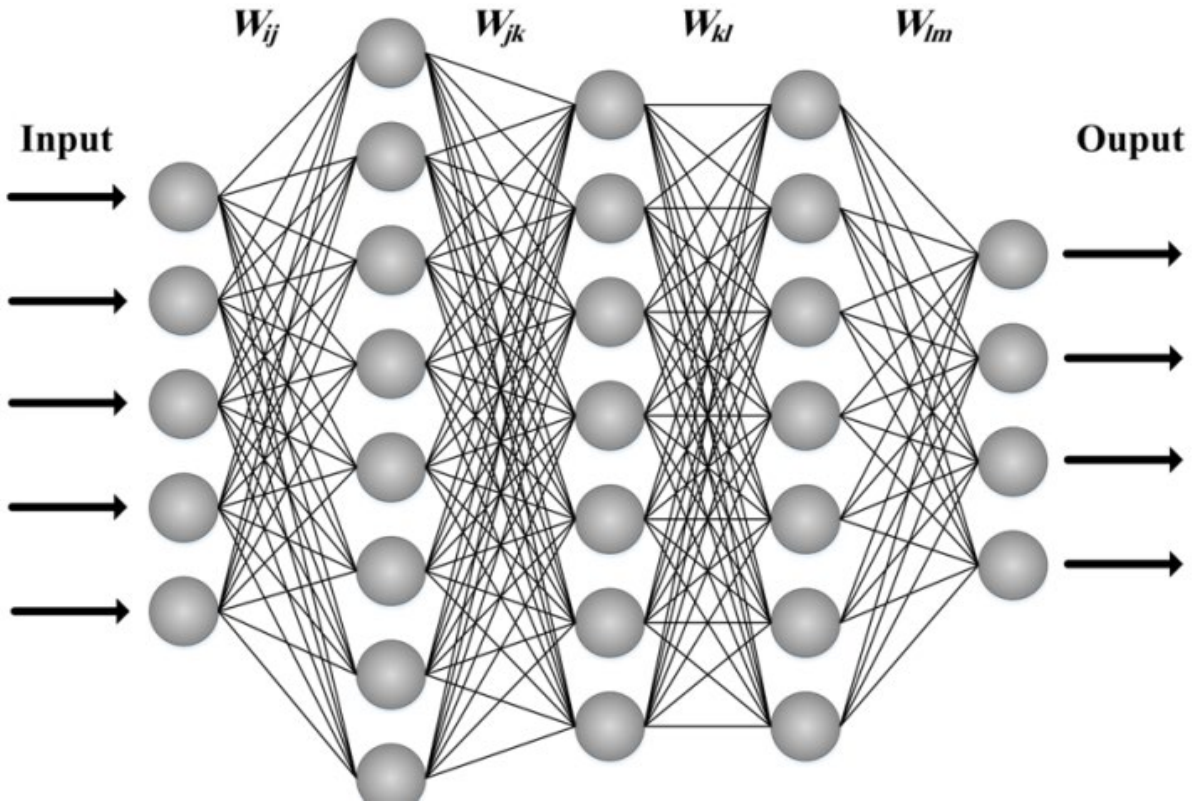
a couple of giraffe standing next to each other

Source: https://cs.stanford.edu/people/karpathy/neuraltalk2/demo.html

MLP for complex Tasks



256 x 256 = 65,536



Limitations with MLP approach

- Model complexity increase with input size.
  - Still require to construct some features by hand.
- Stacking large number of layers will result in
  - Diminishing gradient.
- Sigmoid activations lead to saturation.
- Large connections of the nodes that are unnecessary.

Does not Scale well with the complexity of task.

MLP Limitations

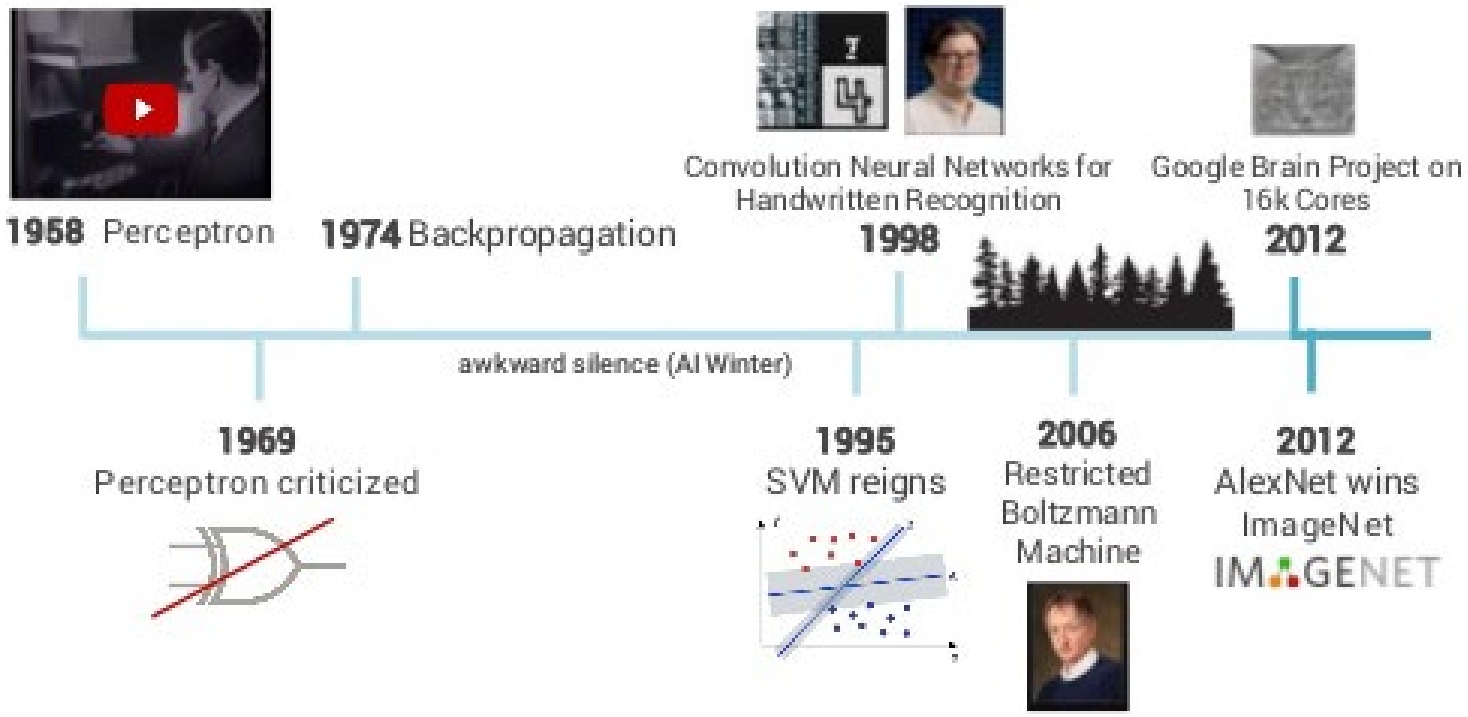
Up until the success of Deep Neural Networks, ML is significantly limited by:

- Features/Attributes of the Data set
- Size of the Data set

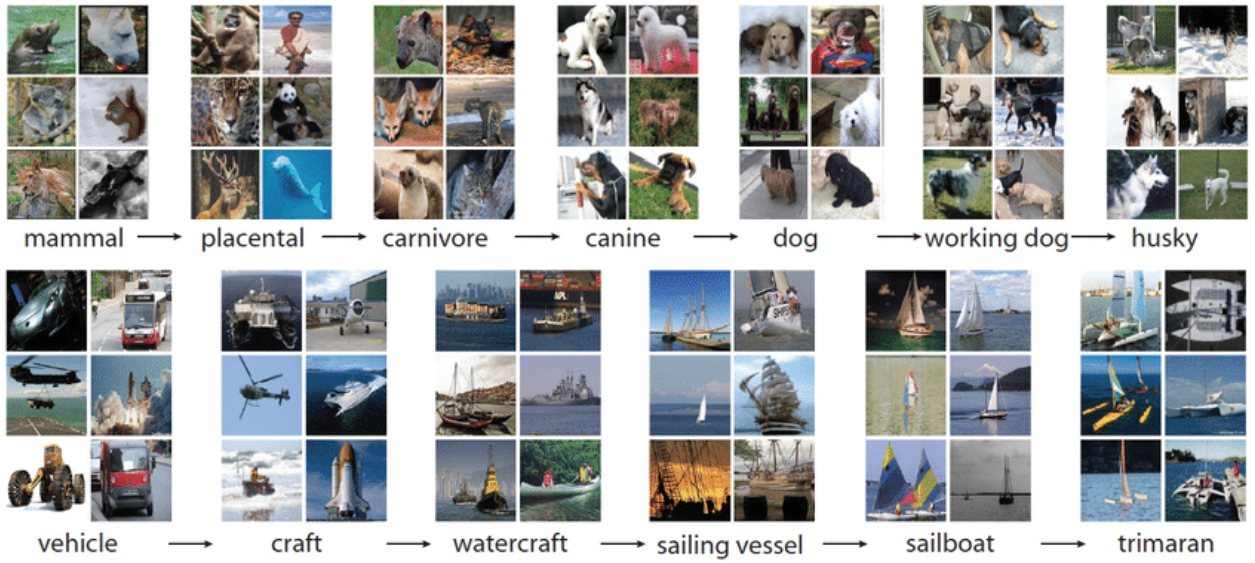
Goal: Eliminate (or automate) the feature engineering



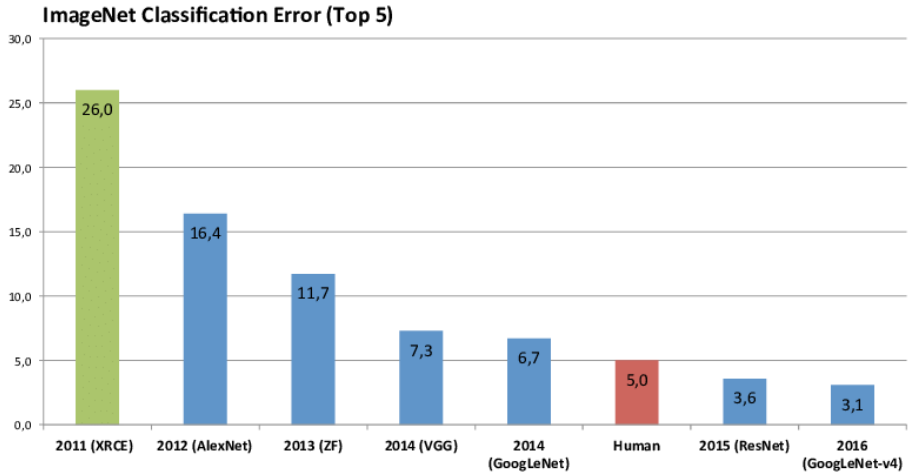
History to Deep Nets



ImageNet Large Scale Visual Recognition Challenge



http://www.image-net.org/challenges/LSVRC/



Deep Neural Networks

A ***deep neutral network*** is a neural network with a large number of hidden layers

- There is a variety of different deep net structures

One widely used deep net structure is a ***Convolution Neural Network (CNN)***

- At the moment, this is what most people refer to when discussing deep learning
- Very successful for image-based, voice-based ML tasks

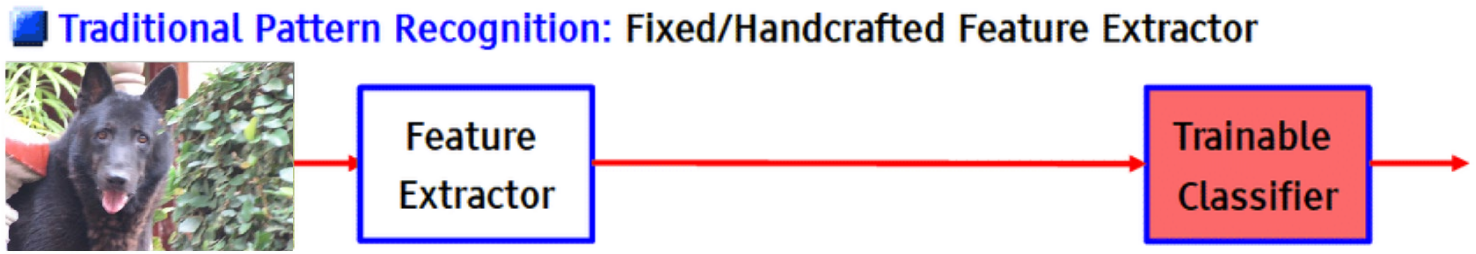
RMIT Classification: Trusted

Computer Vision

Convolution

Feature extraction

Computer Vision



The Feature Extraction:

- Discard redundant information in an image
- Reduce the dimensionality (size) of the image to a manageable scale
- Transform the image into a subset of important or “informative” features
- Use these new features to do actual image recognition

Traditional Computer Vision Pipeline

Before conducting any image recognition/detection a traditional processing pipleline might consist of a variety of steps such as:

- Greyscale conversion
- Normalisation
- Gaussian filtering
- Downsampling
- Edge Detection
- Segmentation





Image Features

Histogram of Gradient HoG (https://www.learnopencv.com/histogram-of-oriented-gradients/)

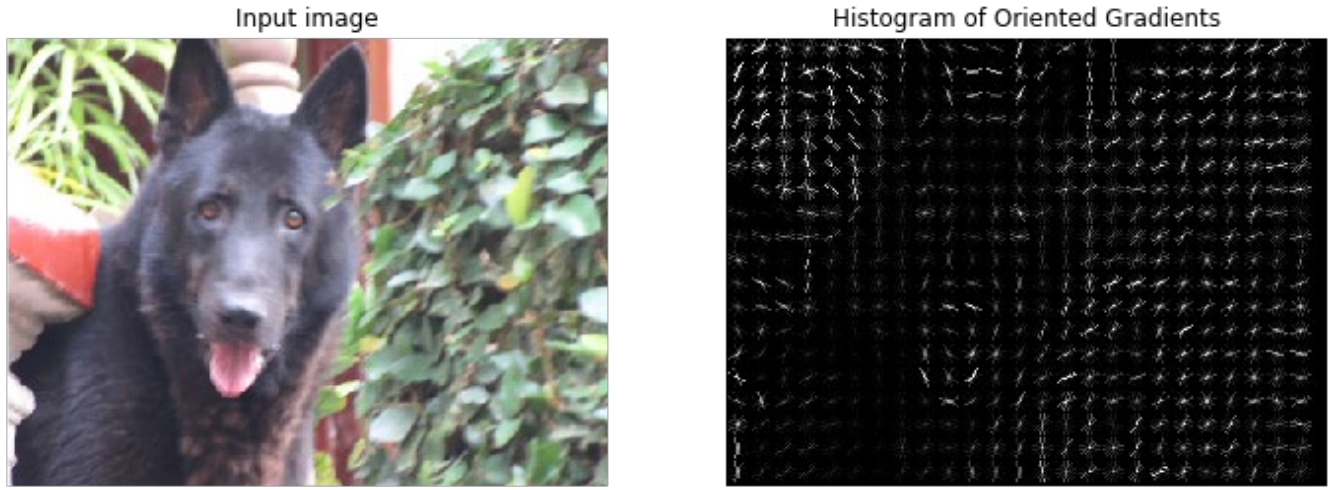
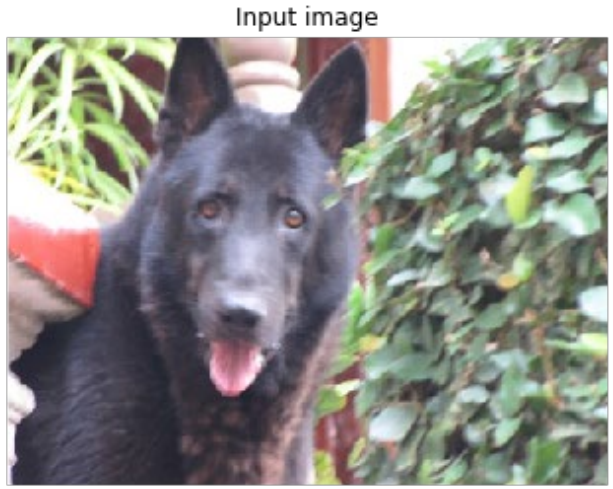


Image Shape: (306, 410, 3) Flattened shape : 376380
HoG feature Shape: 3800

More types of feature extractors: https://scikit-image.org/docs/dev/api/skimage.feature.html

Image Features

Feature construction techniques like PCA can reduce the input dimension further



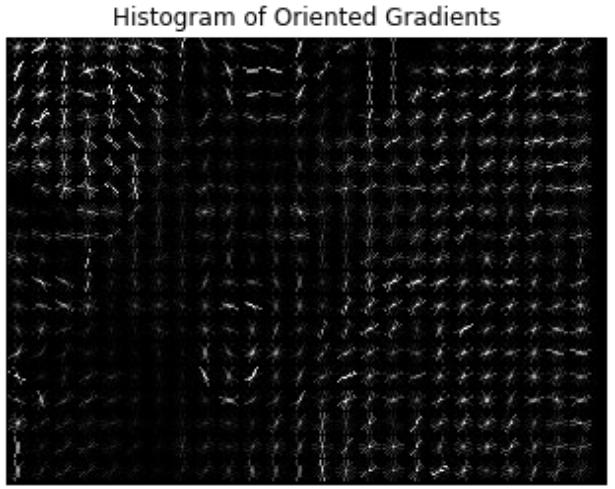


Image Shape: (306, 410, 3) Flattened shape : 376380 HoG feature Shape: 3800

PCA Dim: ?

Image Filters

[0.8	0.74	0.6	0.5	0.47	0.55
[0.81	0.73	0.58	0.45	0.41	0.46
[0.81	0.73	0.58	0.43	0.35	0.38
[0.81	0.73	0.58	0.43	0.31	0.31
[0.82	0.73	0.61	0.44	0.29	0.27
[0.83	0.75	0.62	0.44	0.27	0.25



*

1	0	-1
2	0	-2
1	0	-1

Sobal Filter
(Kernel)

→

-0.89	-1.09	-0.71	0.02	
-0.92	-1.18	-0.91	-0.2	
-0.9	-1.2	-1.11	-0.44	
-0.87	-1.21	-1.26	-0.63	

Image Filters

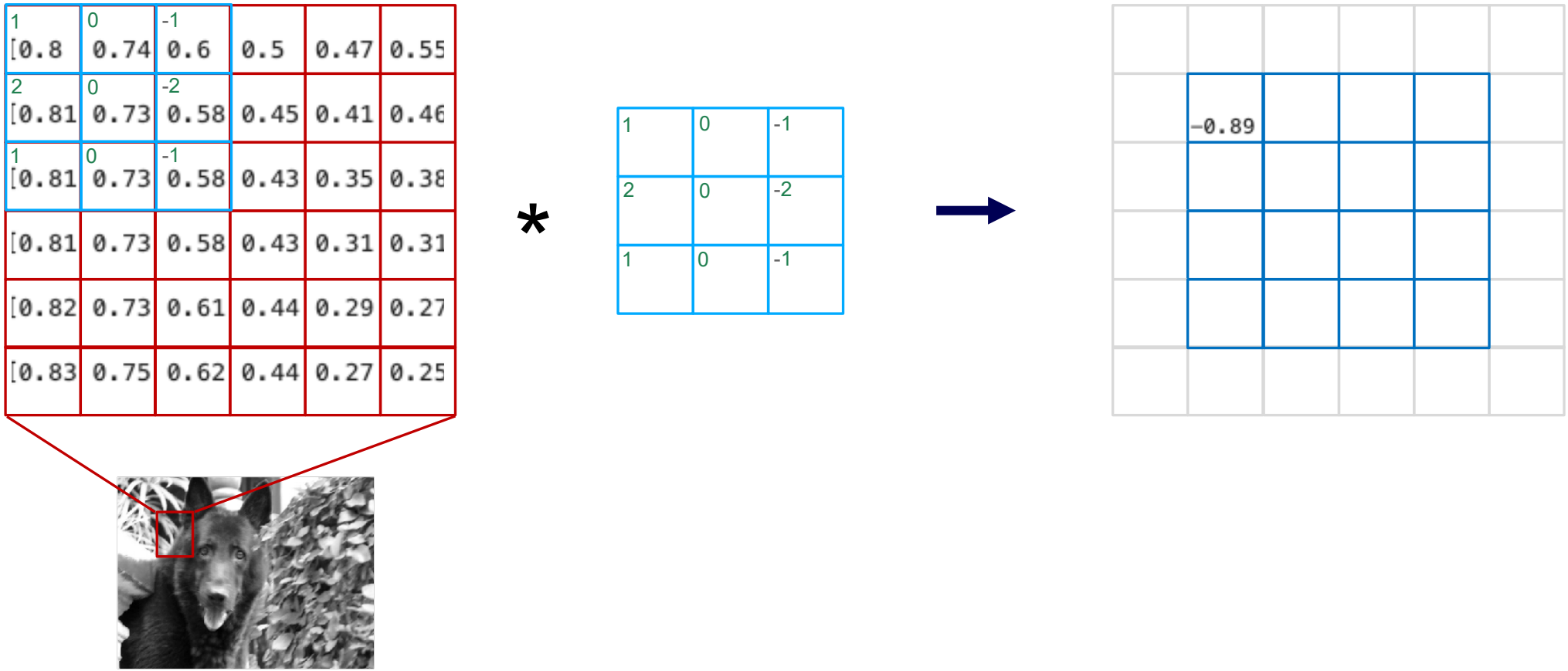


Image Filters

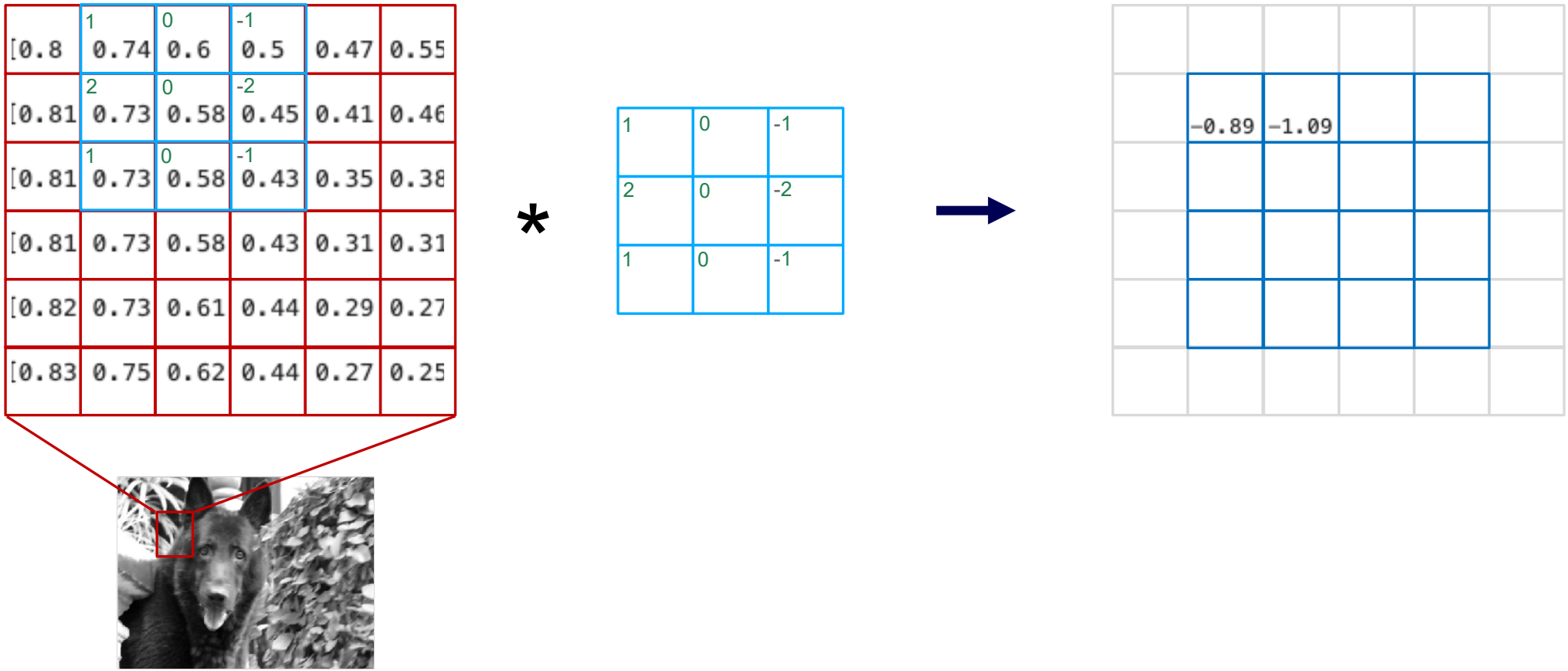


Image Filters

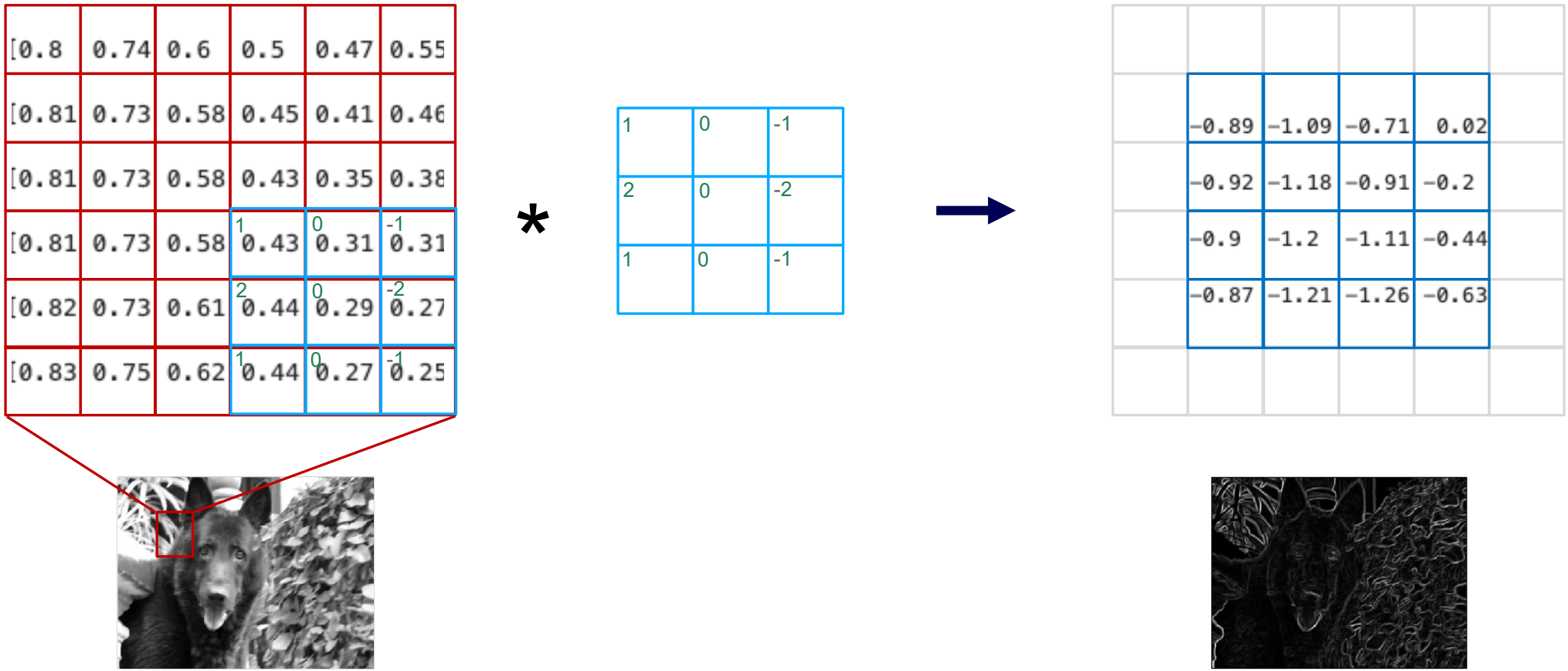


Image Filters

0	0	0	0	0	0	0	0
0	[0.8	0.74	0.6	0.5	0.47	0.55	0
0	[0.81	0.73	0.58	0.45	0.41	0.46	0
0	[0.81	0.73	0.58	0.43	0.35	0.38	0
0	[0.81	0.73	0.58	0.43	0.31	0.31	0
0	[0.82	0.73	0.61	0.44	0.29	0.27	0
0	[0.83	0.75	0.62	0.44	0.27	0.25	0
0	0	0	0	0	0	0	0

Padding

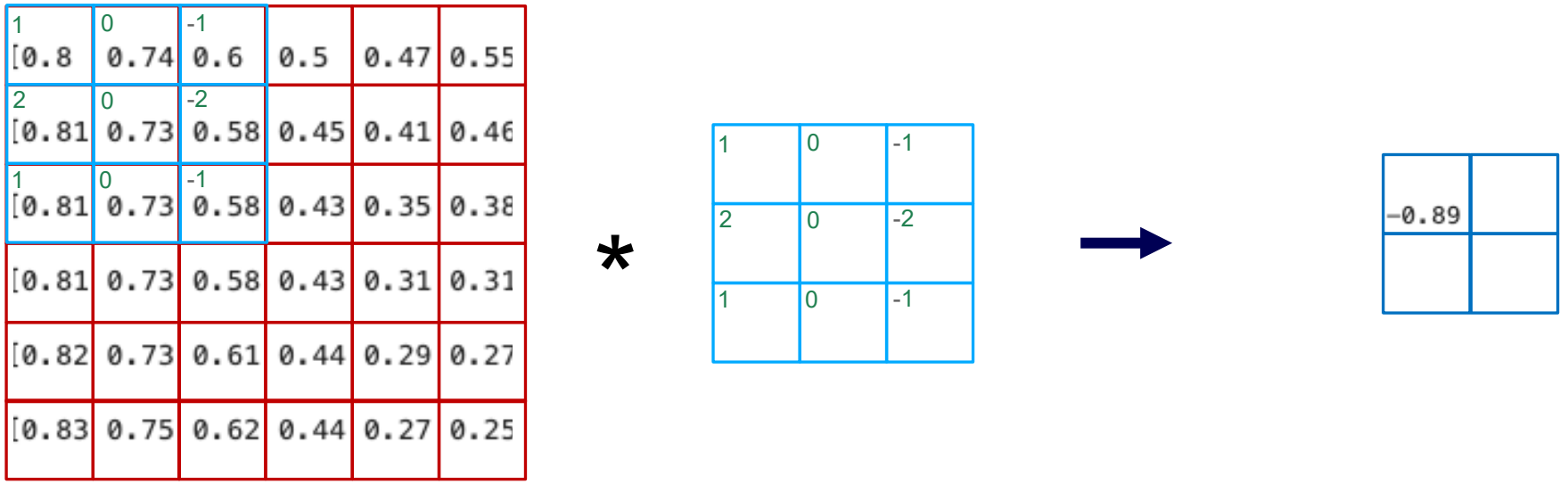
*

1	0	-1
2	0	-2
1	0	-1

2.2	-0.62	-0.75	-0.43	0.11	-1.36
2.92	-0.89	-1.09	-0.71	0.02	-1.64
2.92	-0.92	-1.18	-0.91	-0.2	-1.41
2.93	-0.9	-1.2	-1.11	-0.44	-1.25
2.95	-0.87	-1.21	-1.26	-0.63	-1.16
2.24	-0.64	-0.93	-1.01	-0.54	-0.84

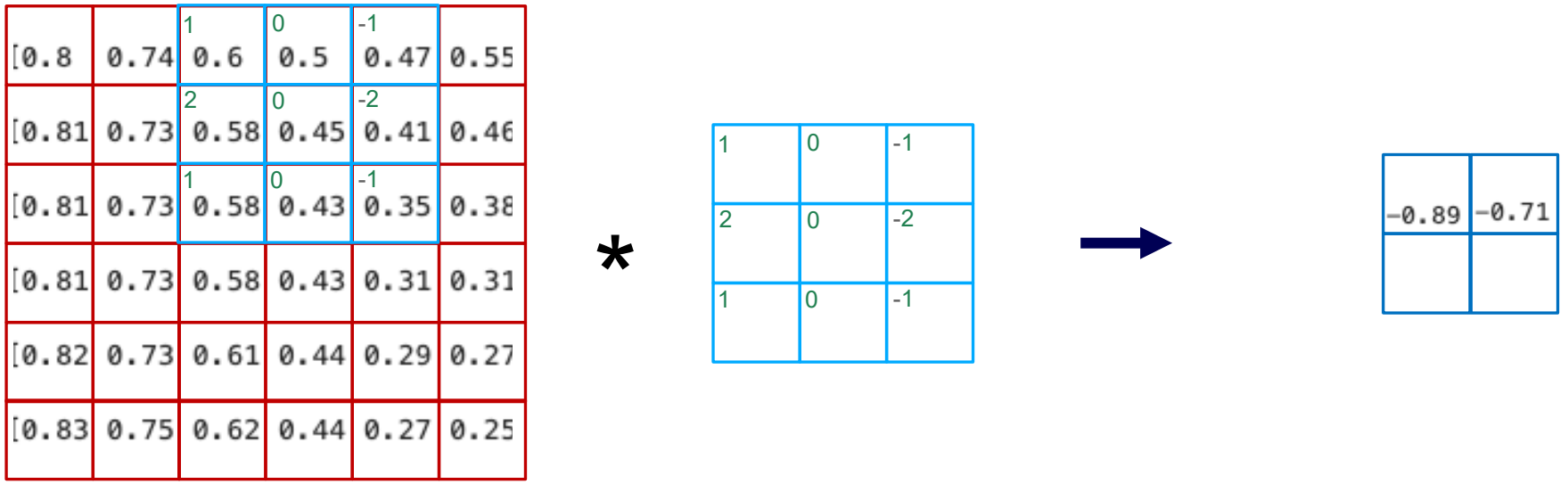


Image Filters



Striding

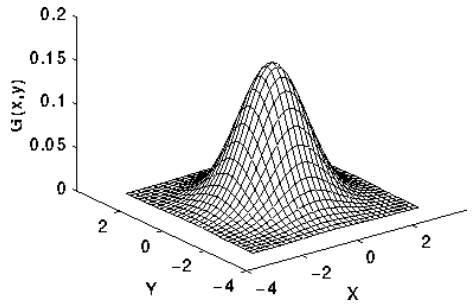
Image Filters



Striding

Image Filters

Gaussian Blur



$$\begin{bmatrix} -2.7778 & -1.3889 & -2.7778 \\ -1.3889 & 0 & -1.3889 \\ -2.7778 & -1.3889 & -2.7778 \end{bmatrix}$$



Original Image



Sigma = 1



Sigma = 10

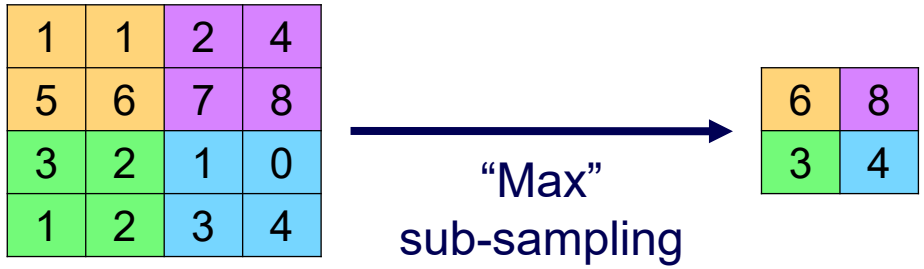
Sub-Sampling

Sub-Sampling reduces the size (dimension/number of pixels) in an image

- The pixels of the downsampled image are derived from the original image

There are many algorithms for downsampling with varying levels of complexity

- Simple nearest neighbour
- Interpolation
- Fourier Transforms
- Vectorisation

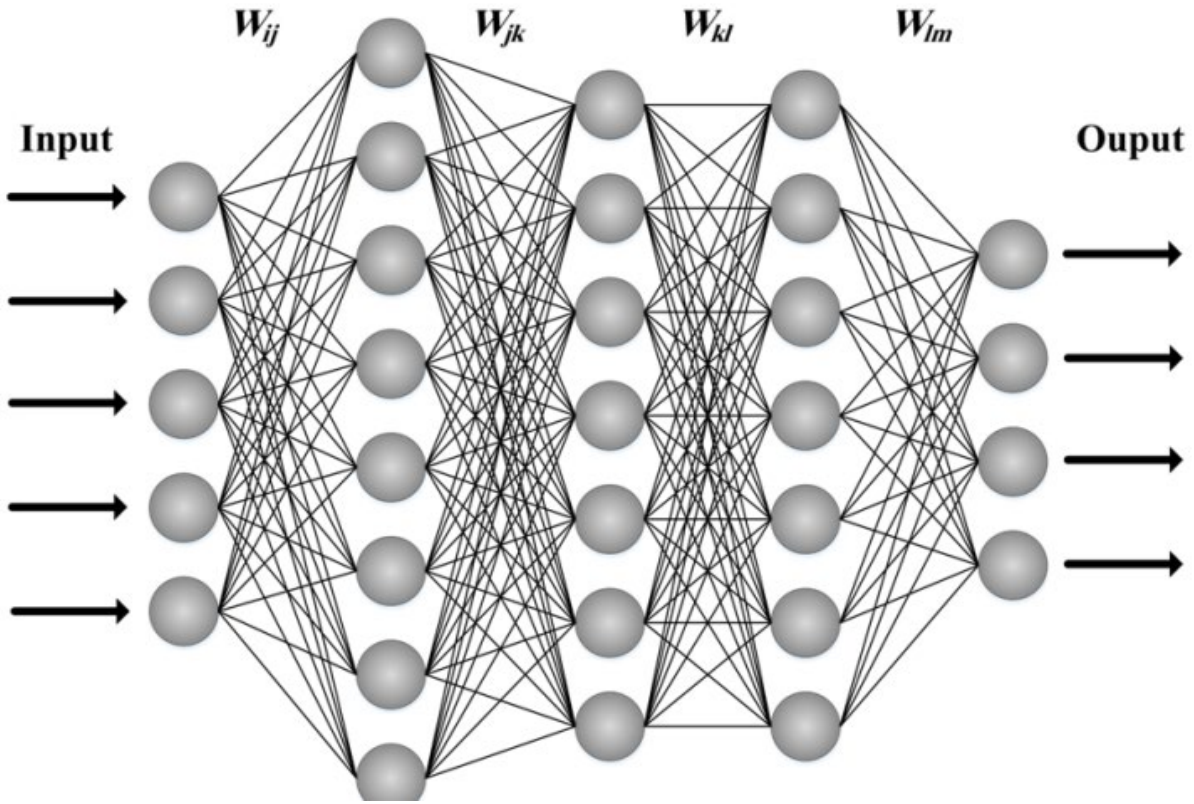


RMIT Classification: Trusted

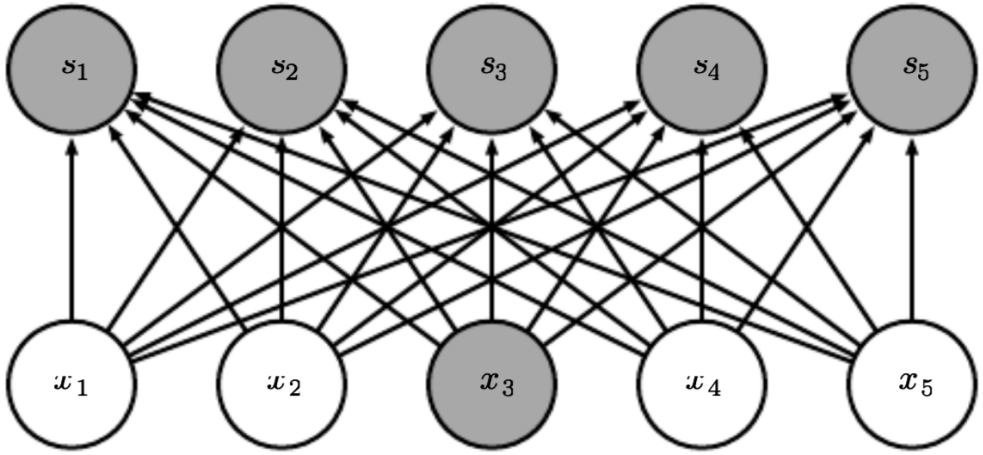
MLP to CNN

MLP for complex Tasks



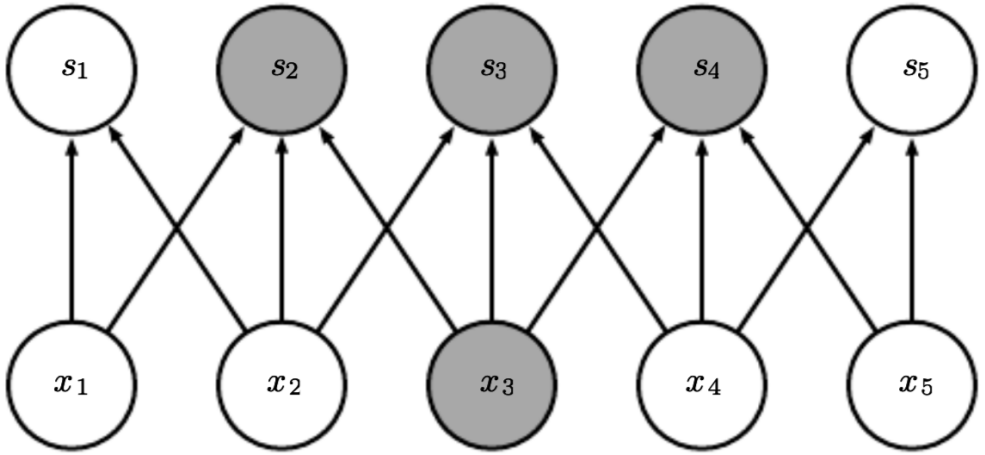


Sparse Connectivity



Fully connected

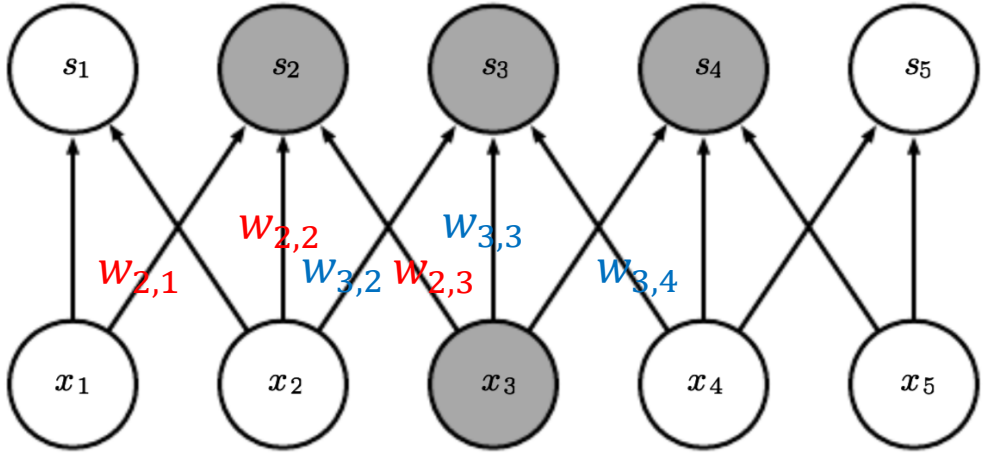




Sparse connected

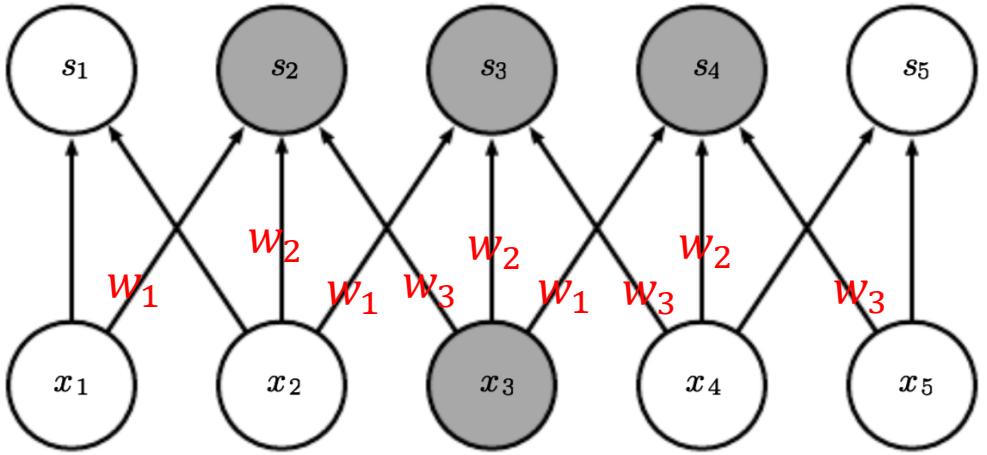
Image: Deep learning, Ian Goodfellow

Parameter Sharing



Sparse connected





Shared weights

Image: Deep learning, Ian Goodfellow

Learning Convolution Filters

[0.8	0.74	0.6	0.5	0.47	0.55
[0.81	0.73	0.58	0.45	0.41	0.46
[0.81	0.73	0.58	0.43	0.35	0.38
[0.81	0.73	0.58	0.43	0.31	0.31
[0.82	0.73	0.61	0.44	0.29	0.27
[0.83	0.75	0.62	0.44	0.27	0.25



*

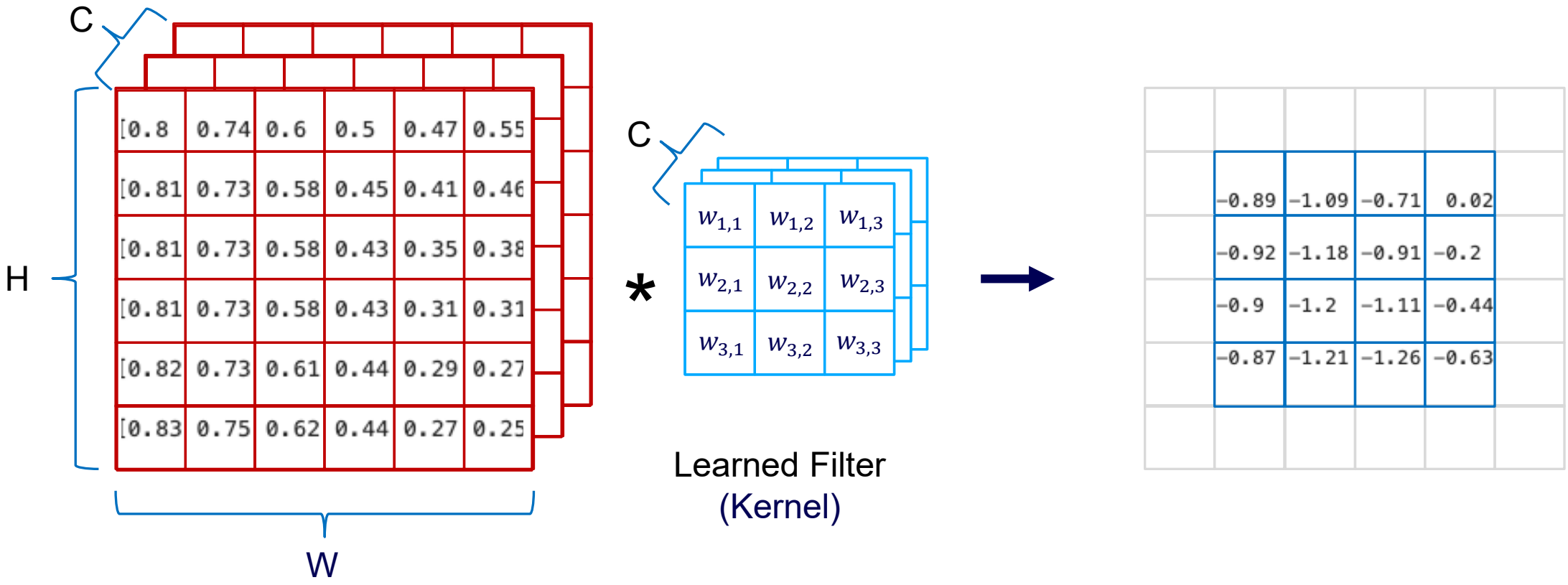
$w_{1,1}$	$w_{1,2}$	$w_{1,3}$
$w_{2,1}$	$w_{2,2}$	$w_{2,3}$
$w_{3,1}$	$w_{3,2}$	$w_{3,3}$

Learned Filter
(Kernel)

→

-0.89	-1.09	-0.71	0.02
-0.92	-1.18	-0.91	-0.2
-0.9	-1.2	-1.11	-0.44
-0.87	-1.21	-1.26	-0.63

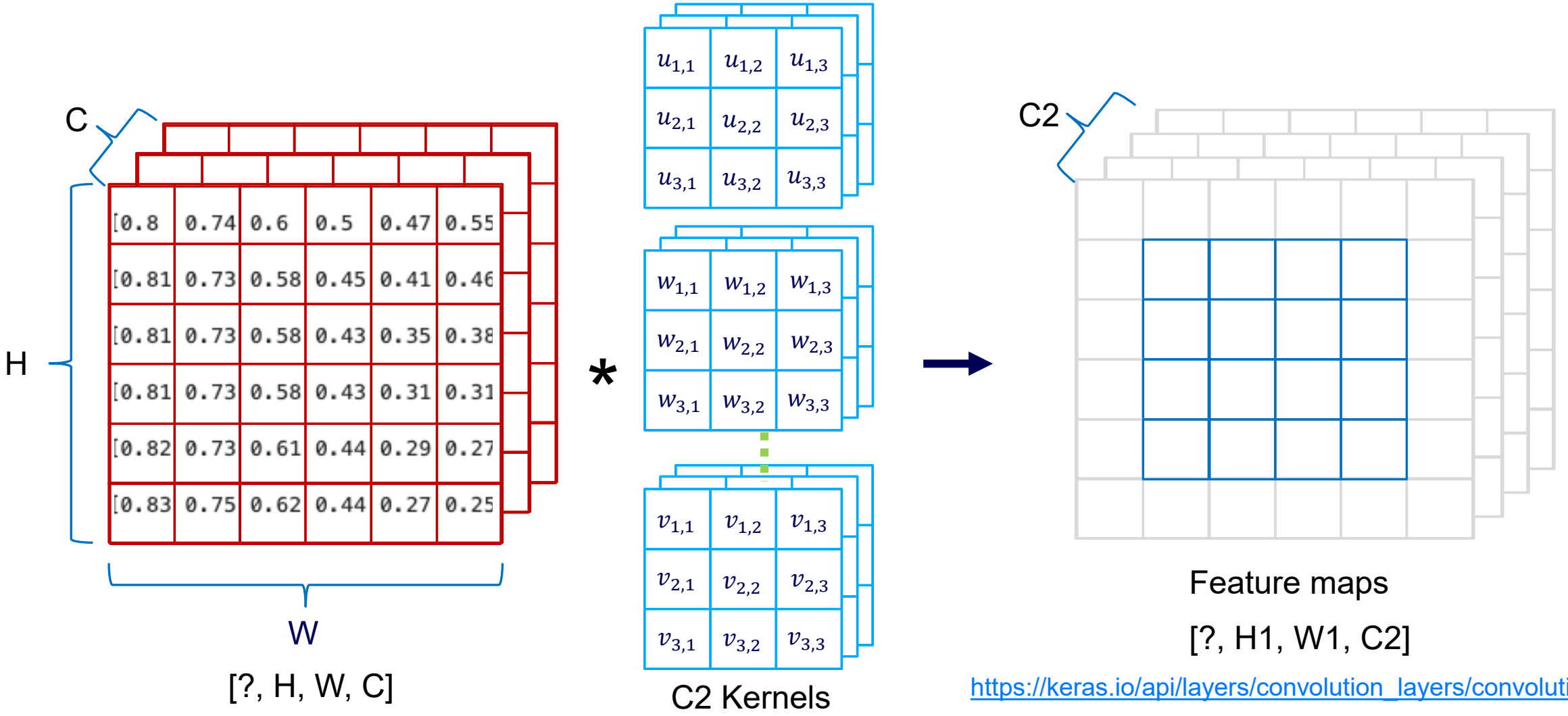
Learning Convolution Filters



[?, H, W, C]

[?, H1, W1, 1]

Learning Convolution Filters



https://keras.io/api/layers/convolution_layers/convolution2d/

Feature Map

One filter is not sufficient, so the convolution has multiple filters

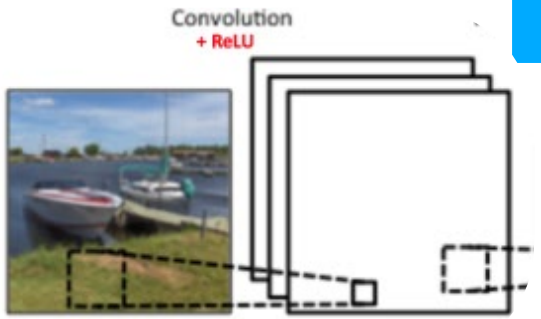
These filters form a ***feature map***:

- The weights of *each* filter will be learnt when the CNN is trained
- All perceptrons for the *same filter* use the *same weights*

The feature map has many tuneable parameters:

- Size: Width and height of the filter
- Depth: Number of filters in the convolution
- Stride: Distance to move the centre of each filter over the input
  - Traditional image processing has a stride of 1
- Zero-padding: It is sometimes convenient to pad the input matrix with zeros around the border, so that the filter to can applied to bordering elements.

Why Convolutions?



Convolutions are inspired by traditional image processing, however, there are questions:

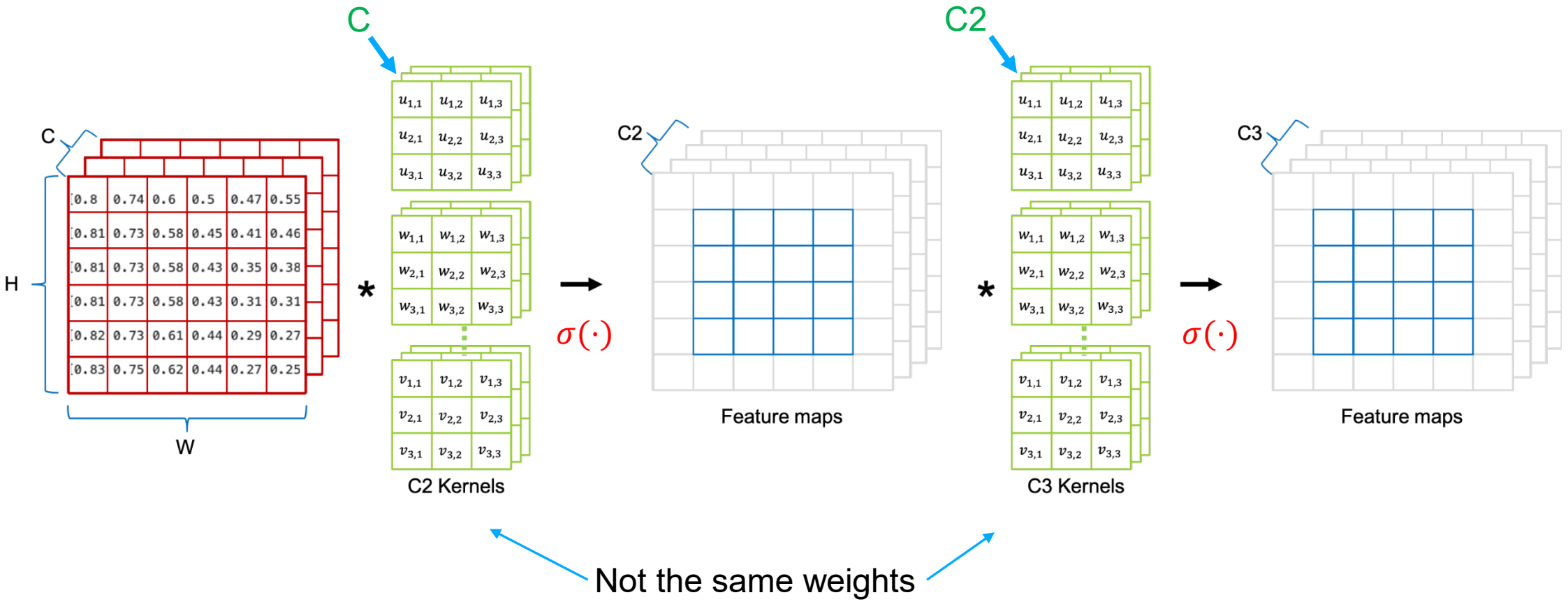
- Would it be better to use a small window of perceptrons, but allow all of the weights to be different?
- Why, in practice, are convolutions useful/good?

Convolutions provide benefits over “traditionally connected perceptrons” because:

- Parameter sharing: A feature detector (such as a vertical edge detector) that’s useful in one part of the image is probably useful in another part of the image.
- Sparsity of connections: In each layer, each output value depends only on a small number of inputs.
- Training speed: With significantly fewer weights, training is significantly faster

Learning Convolution Filters

Stacked Convolution layers - Activations



Activation Function

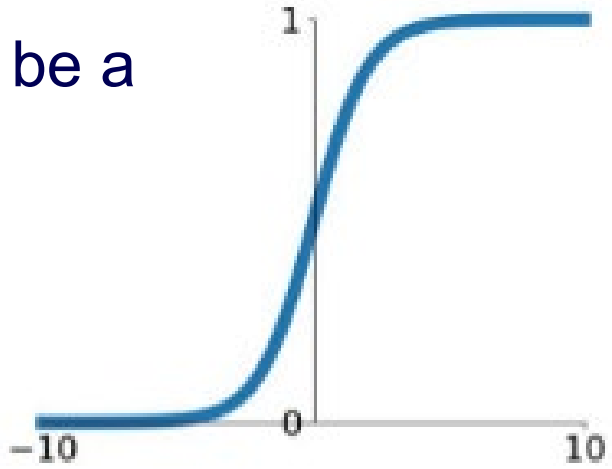
In the convolutional layer, so far, we have assumed a standard perceptron:

- This uses the sigmoid function (or tanh function)

In practice, training (and feed-forward calculations) with a sigmoid function are slow, especially for deep-nets with a large number of layers

- The exponential curve is computationally slow to compute
- Lead to saturation
- However, we still need ***non-linearity*** in the perceptrons

In practice, an alternative activation function has proved to be a suitable balance as an activation function for CNNs



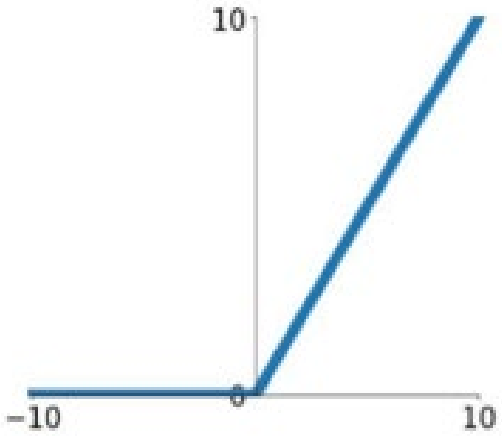
Rectified Non Linearity (ReLU)

Rectified Linear Unit (ReLU) is a non-linear operation where:

- It is an element wise operation (applied per pixel)
- Negative values are truncated to 0 (rectified)
- Positive values are untouched (linear mapping)

ReLU is used in practice because:

- They propagate the gradient efficiently and reduce the likelihood of a vanishing gradient
- Address issues of the cancellation problem, and result in a more sparse activations. The sparsity can be robust to small changes in the input, such as noise
- Computationally efficient - AlexNet (2012) operates up to 6x faster

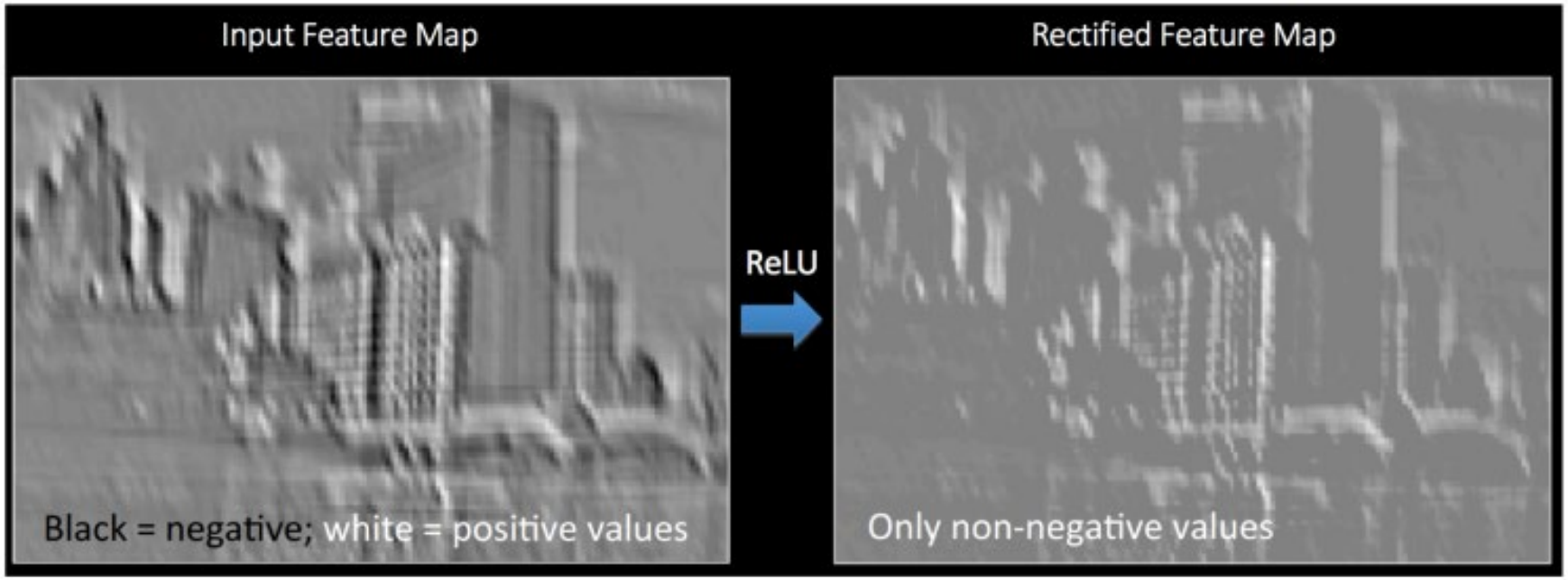


ReLU

$$\max(0, x)$$

Rectified Non Linearity (ReLU)

ReLU operation applied to one an example feature map



**Technical Note

In these notes, the CNN structure is correlated to the classic perceptron structure

In current practice, and literature, this is not done. Instead:

The Convolution is treated simply as a matrix filter

The ReLU is treated as another input map / matrix operation / layer

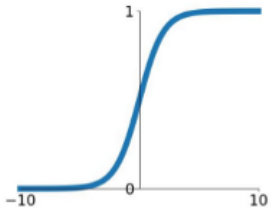
These makes the components matrix operations which are:

- Highly parallel multiplication operations
- Significantly more computationally efficient
- Implemented on GPU hardware

Other Activation Functions

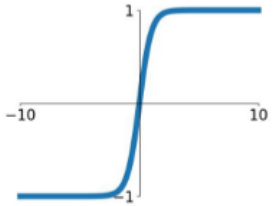
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



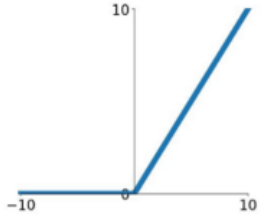
tanh

$$\tanh(x)$$



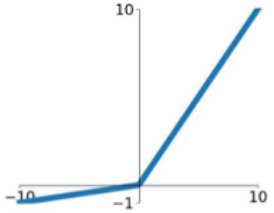
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

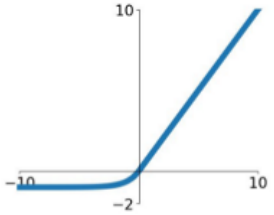


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

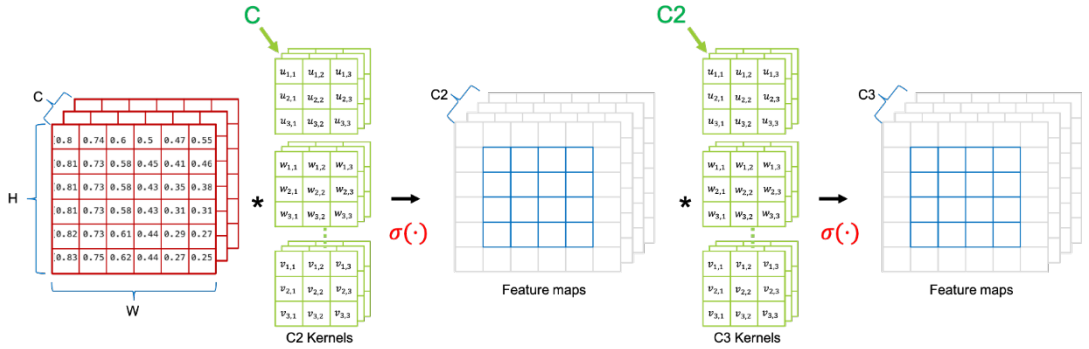
ELU

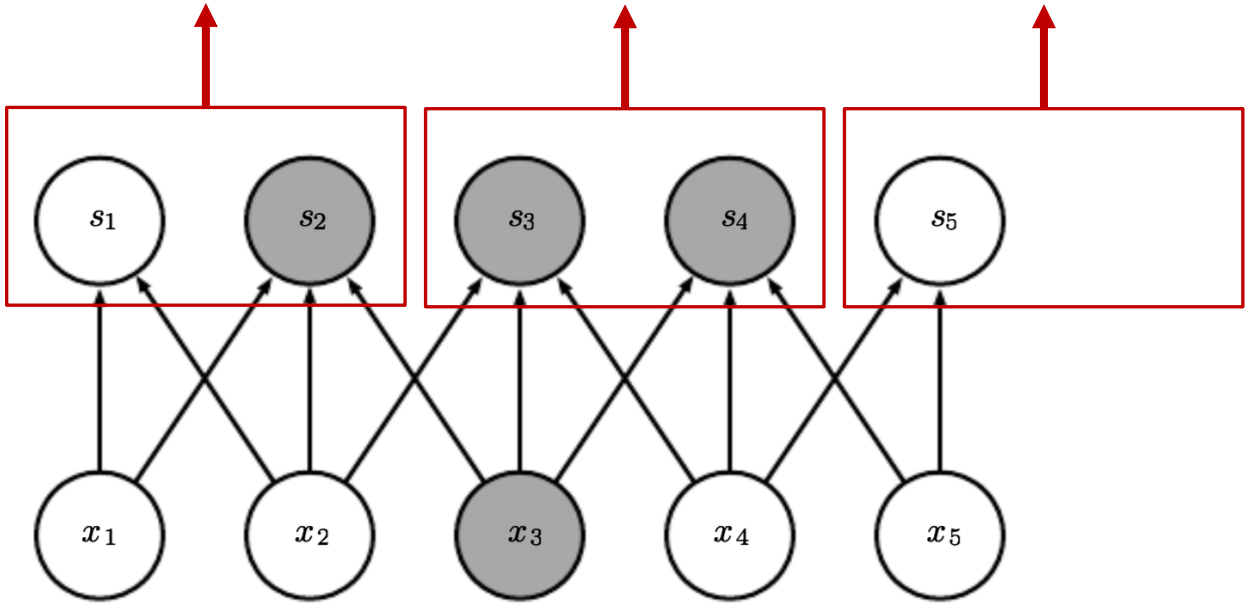
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



https://keras.io/api/layers/activations/

Pooling


C
H
W
C2 Kernels
$\sigma(\cdot)$
C2
Feature maps
C2
C3 Kernels
$\sigma(\cdot)$
C3
Feature maps

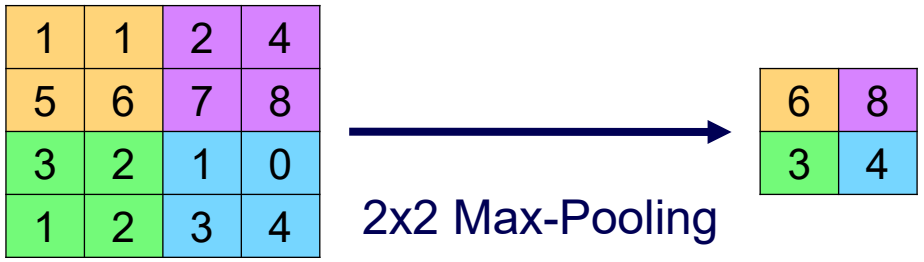

s_1
s_2
s_3
s_4
s_5
x_1
x_2
x_3
x_4
x_5

Pooling

Pooling is sub-sampling:

- Reduces the dimensionality of each feature map
- Applied to each feature map independently

Pooling is one of three typical simple matrix operations: Max, Average, Sum



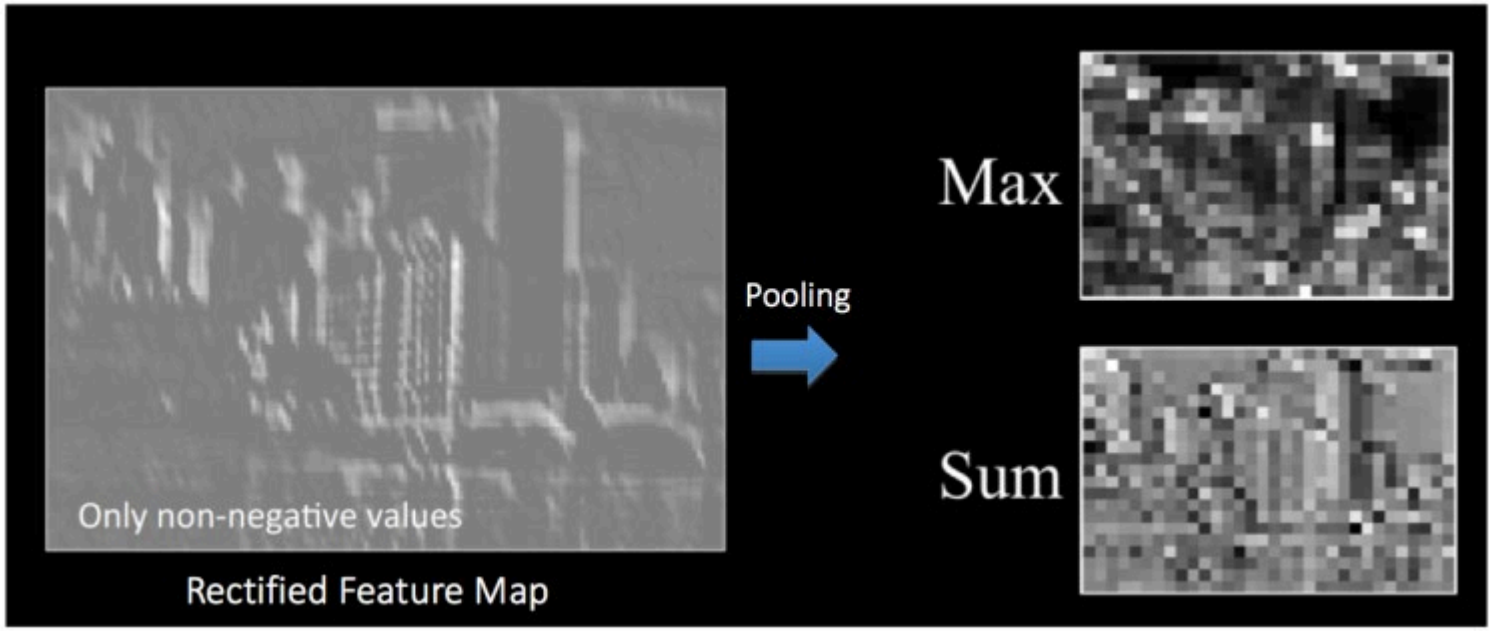
Pooling

The purpose of Pooling is:

- Make the input representations (feature dimension) smaller and more manageable
- Reduce the number of parameters in the network, helping to limit overfitting
- Make the network invariant to small transformations, scaling distortions and translations in the image

The Pooling Step

Example of Pooling on a rectified feature map



RMIT Classification: Trusted

CNN Architecture

CNN Building-Blocks

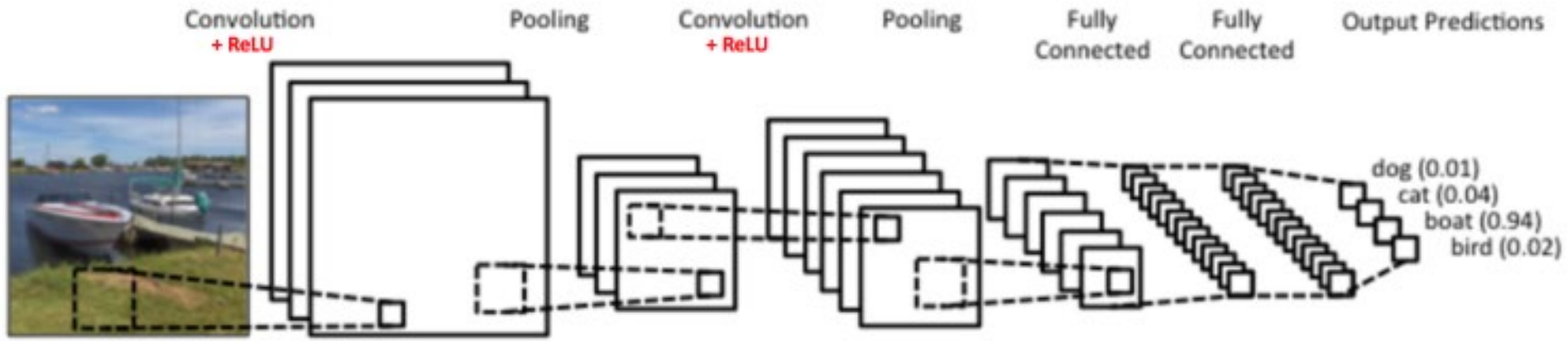
Now we have Learned about **4 main building blocks** of CNNs

- Convolutions (2D)
- Pooling (or sub-sampling)
- Non-Linearity (Activations)
- Fully Connected MLP

LeNet Architecture (1990s)

The LeNet CNN is considered the classic CNN architecture that demonstrated a successful Deep network design for image recognition.

- It contains a “relatively simple” combination of the 4 key CNN components



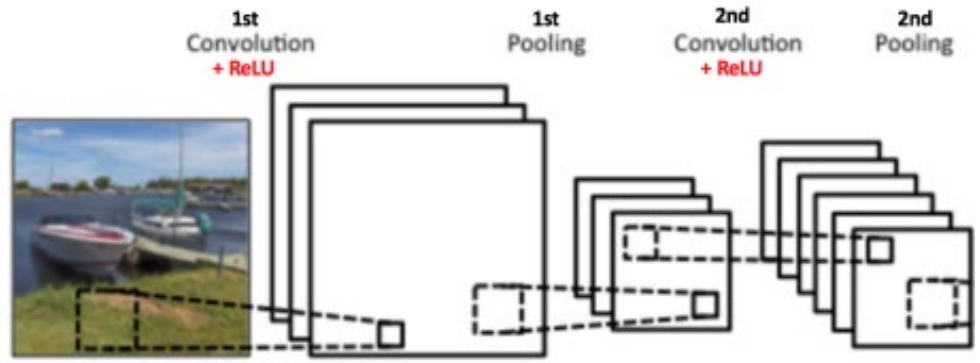
Repeated Convolutions & Pooling

In a typical CNN, multiple convolution and pooling layers are used.

The number of filters may be varied between each convolution

- Each feature map may be treated individually by subsequent convolutions, or
- A convolution may be applied *over* multiple feature maps

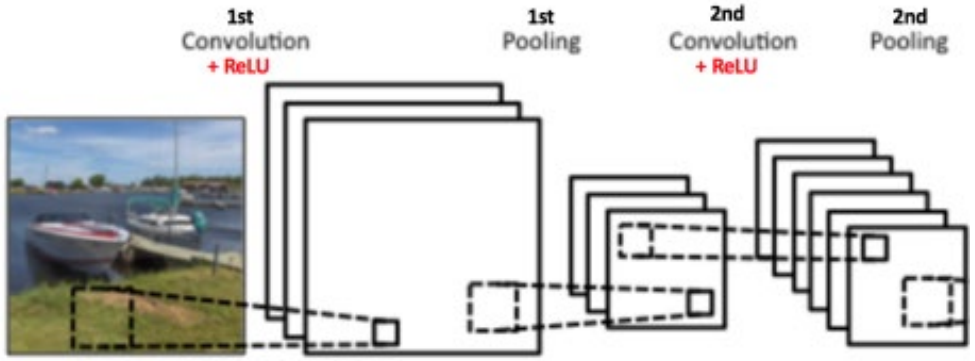
There are a large number of design variations for how to iterate over multiple layers



Repeated Convolutions & Pooling

In a typical CNN, multiple convolution and pooling layers are used.

- Different pooling methods may also be used
- Pooling may be discarded for later convolutions



Fully Connected Layer

The last stage of a CNN is a traditional fully connected layer:

- That is, an MLP
- For classification a softmax activation function in the output layer.

The number of layers in the MLP may be varied

- Typically between 2-3 layers are used

Architecture (Modified)

Define Structure

Initialize Computation graph

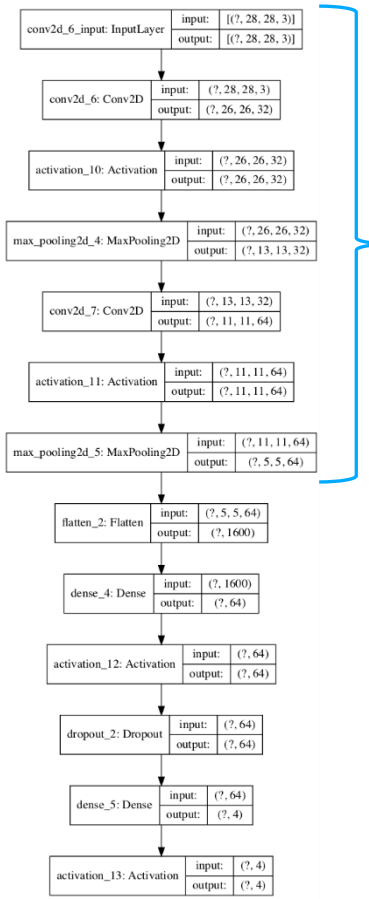
```
model_cnn = Sequential()
model_cnn.add(Conv2D(32, (3, 3),kernel_regularizer=regularizers.l2(0.001), input_shape=(28, 28, 3)))
model_cnn.add(Activation('relu'))
model_cnn.add(MaxPooling2D(pool_size=(2, 2)))

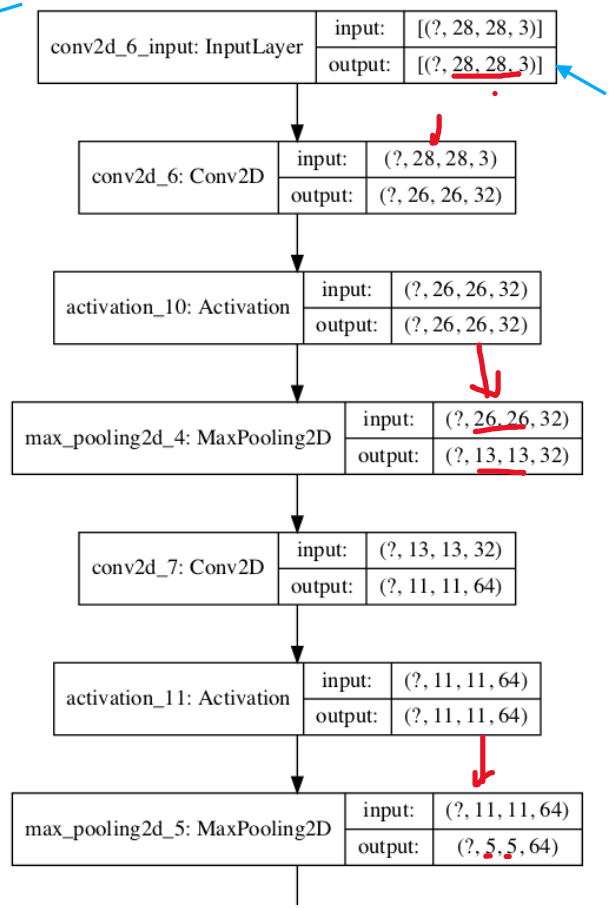
model_cnn.add(Conv2D(64, (3, 3)))
model_cnn.add(Activation('relu'))
model_cnn.add(MaxPooling2D(pool_size=(2, 2)))

model_cnn.add(Flatten())  # this converts our 3D feature maps to 1D feature vectors
model_cnn.add(Dense(64))
model_cnn.add(Activation('relu'))
model_cnn.add(Dropout(0.5))
model_cnn.add(Dense(4))
model_cnn.add(Activation('softmax'))

sgd = optimizers.SGD(lr=0.01, decay=1e-6, momentum=0.9, nesterov=True)
model_cnn.compile(loss='categorical_crossentropy',
              optimizer=sgd,
              metrics=[categorical_accuracy])
```

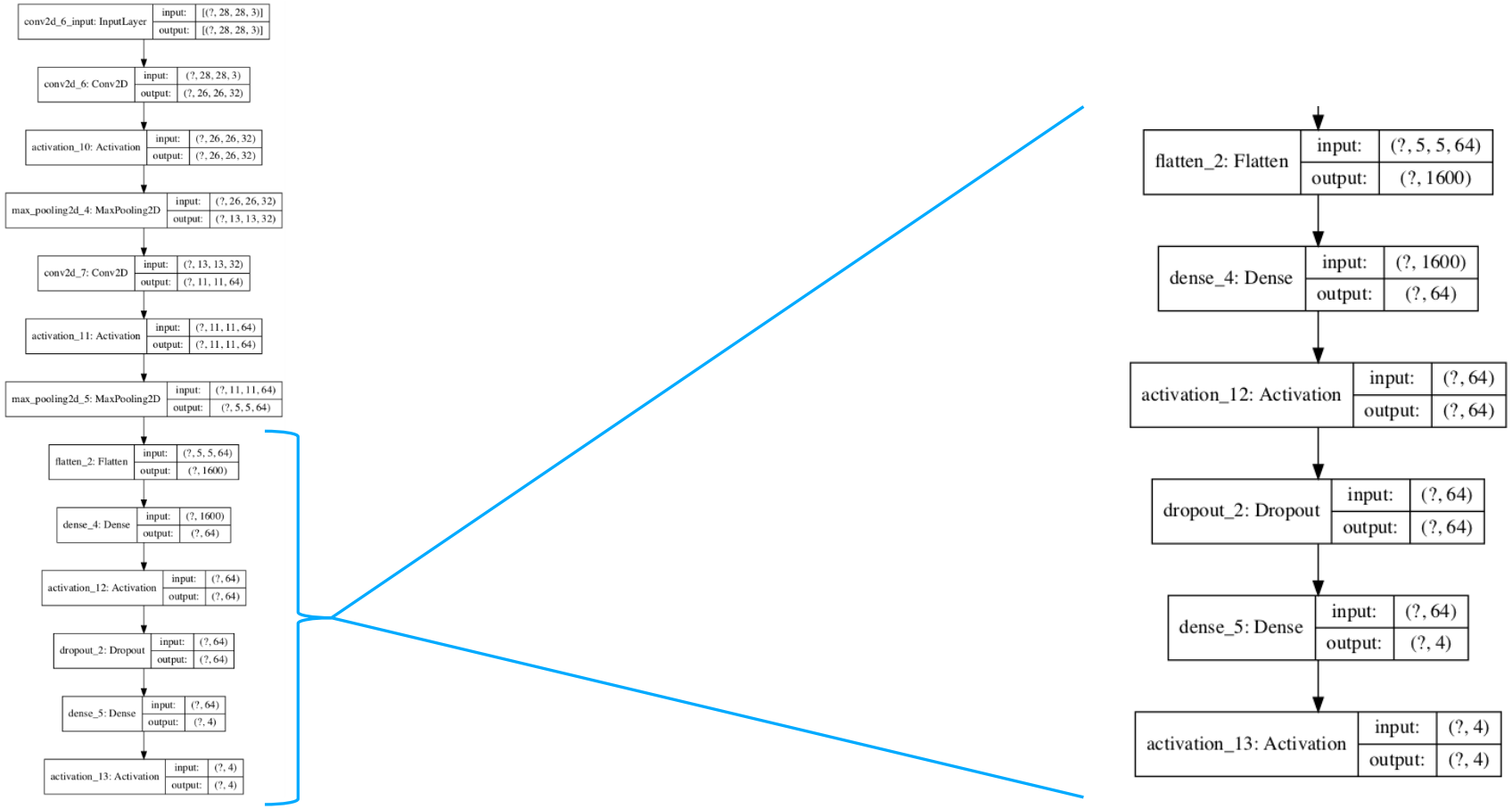
Architecture – Feature Extraction





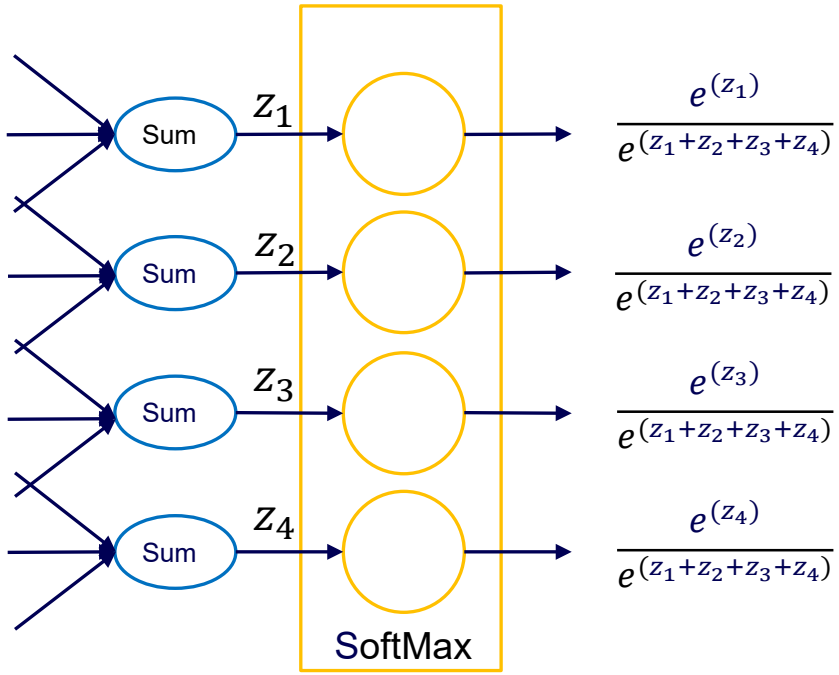
[Batch_size, H, W, Channels]

Architecture - Classification



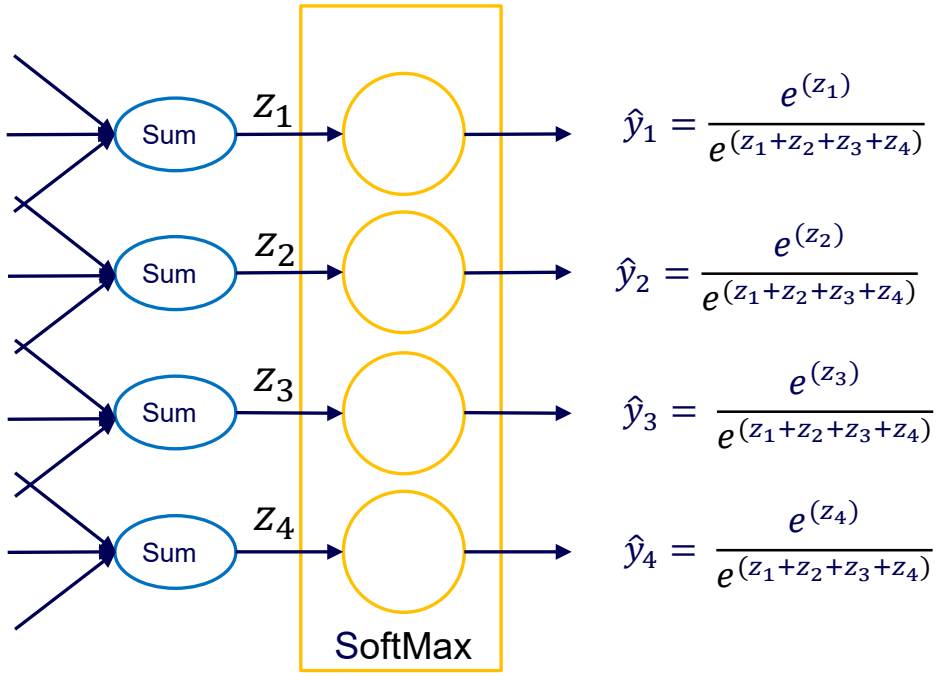
Softmax

If the task is multi-class classification, typically SoftMax activation is used at the output layer.



$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}} \quad \text{for } j = 1, \ldots, K.$$

Categorical Cross-entropy



$$\mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right) = -\sum_{j=0}^{c} y_j^{(i)} \log\left(\hat{y}_j^{(i)}\right)$$

Training with Back-propagation

Training a CNN uses back-propagation and works the same as with an MLP:

Step 1: Initialise all filters/weights with random values

Step 2: Select a batch size, and feed-forward the training instances

Step 3: Calculate the total error at the output layer (summation over all 4 classes)

- Categorical cross-entropy

Step 4: Use back-propagation to calculate the gradients of the error with respect to all weights in the network and use gradient descent to update all filter values / weights.

- The filter update is slightly different for ReLU, but still comes from derivative

Step 5: Repeat until the error is suitably minimised

RMIT Classification: Trusted

What have we learned?

CNN Architecture

The purpose of the image processing pipeline is:

- Discard redundant information in an image
- Reduce the dimensionality (size) of the image to a manageable scale
- Transform the image into a subset of important or “informative” features
- Use these new features to do actual image recognition

After processing, the image is suitable for training with a traditional MLP

The architecture of a Convolutional Neural Network (CNN) mimics this pipeline

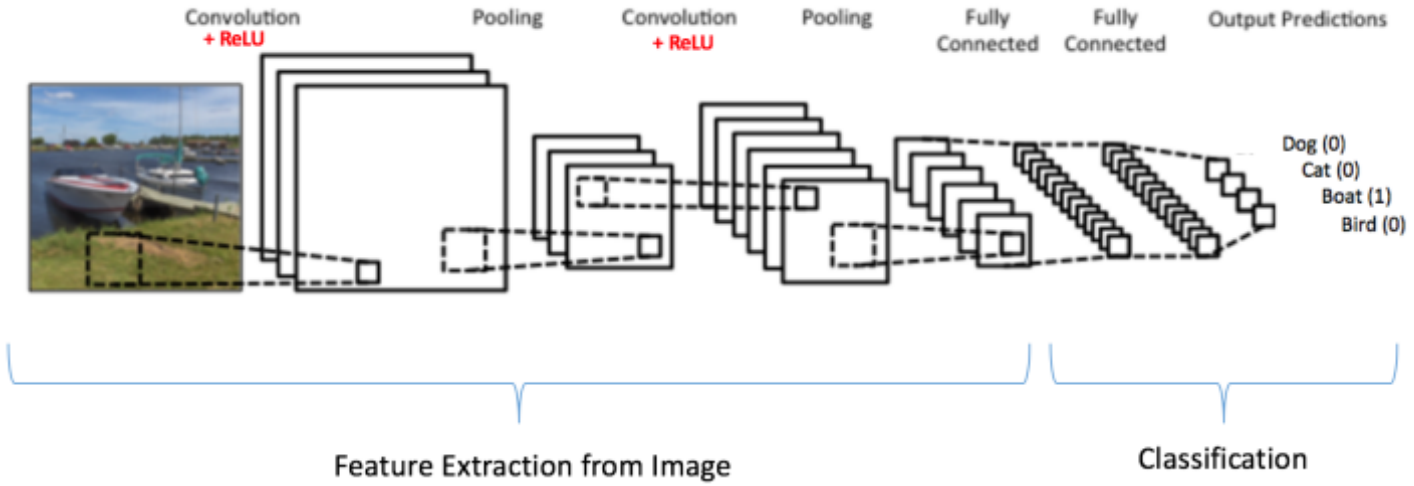
- Many variations of CNNs exist. We will discus the typical elements of a CNN

Final CNN

The final CNN structure effectively gives two parts:

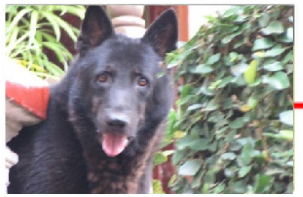
- The convolution conducts ***feature construction*** to transform the input into a suitable attribute space
- ***Classification*** using the constructed/learnt features

Thus a CNN learns both classification and feature construction

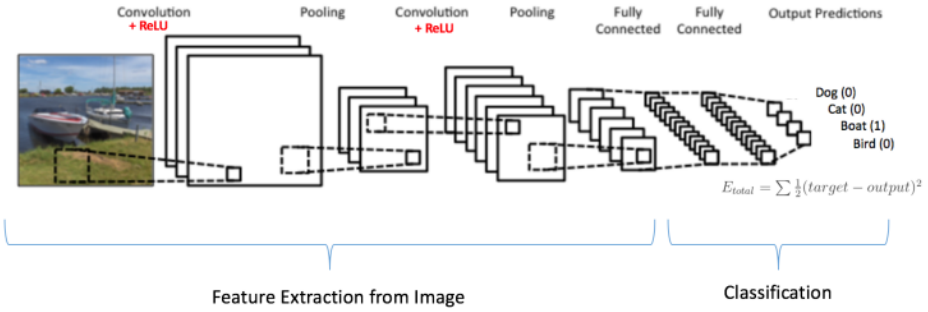


Computer Vision

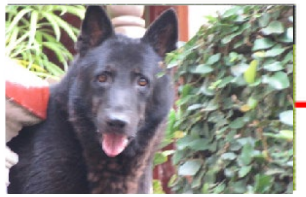
Traditional Pattern Recognition: Fixed/Handcrafted Feature Extractor



Feature Extractor → Trainable Classifier

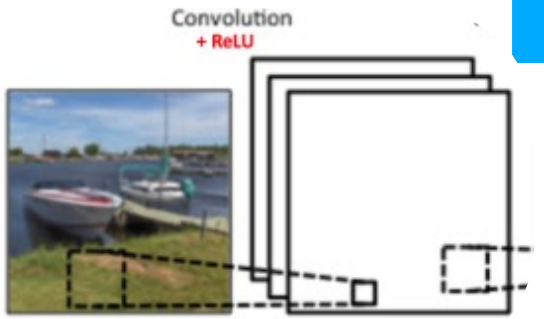


Deep Learning: Representations are hierarchical and trained

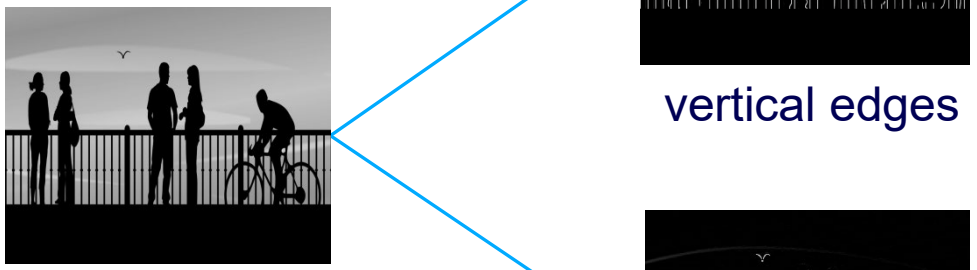


Low-Level Features → Mid-Level Features → High-Level Features → Trainable Classifier

Convolutional Layer



After training, what does a convolutional layer look like?

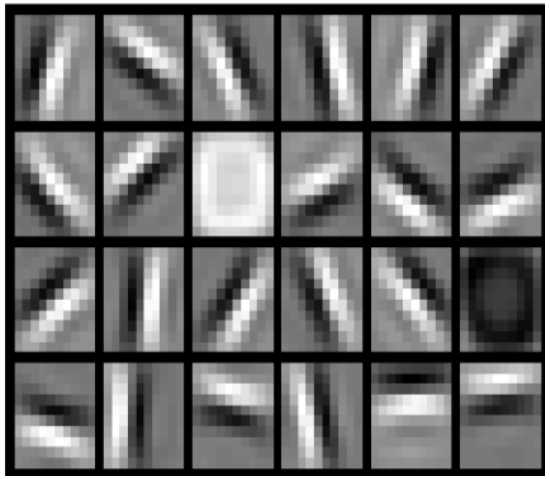




vertical edges



horizontal edges



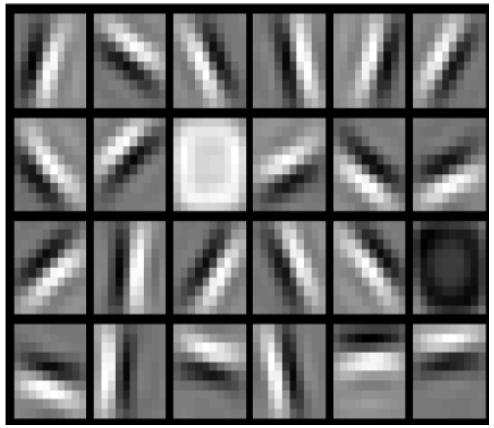
Learned Feature

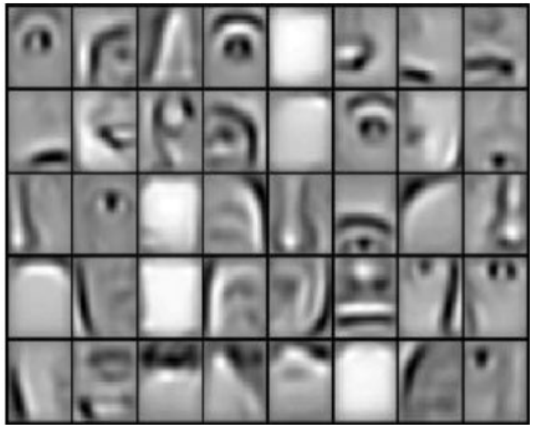
Repeated Convolutions

Multiple convolutional layers effectively construct more complex feature maps

These feature maps are often what people refer to when they discuss what a CNN (or deep net) has learnt/discovered.

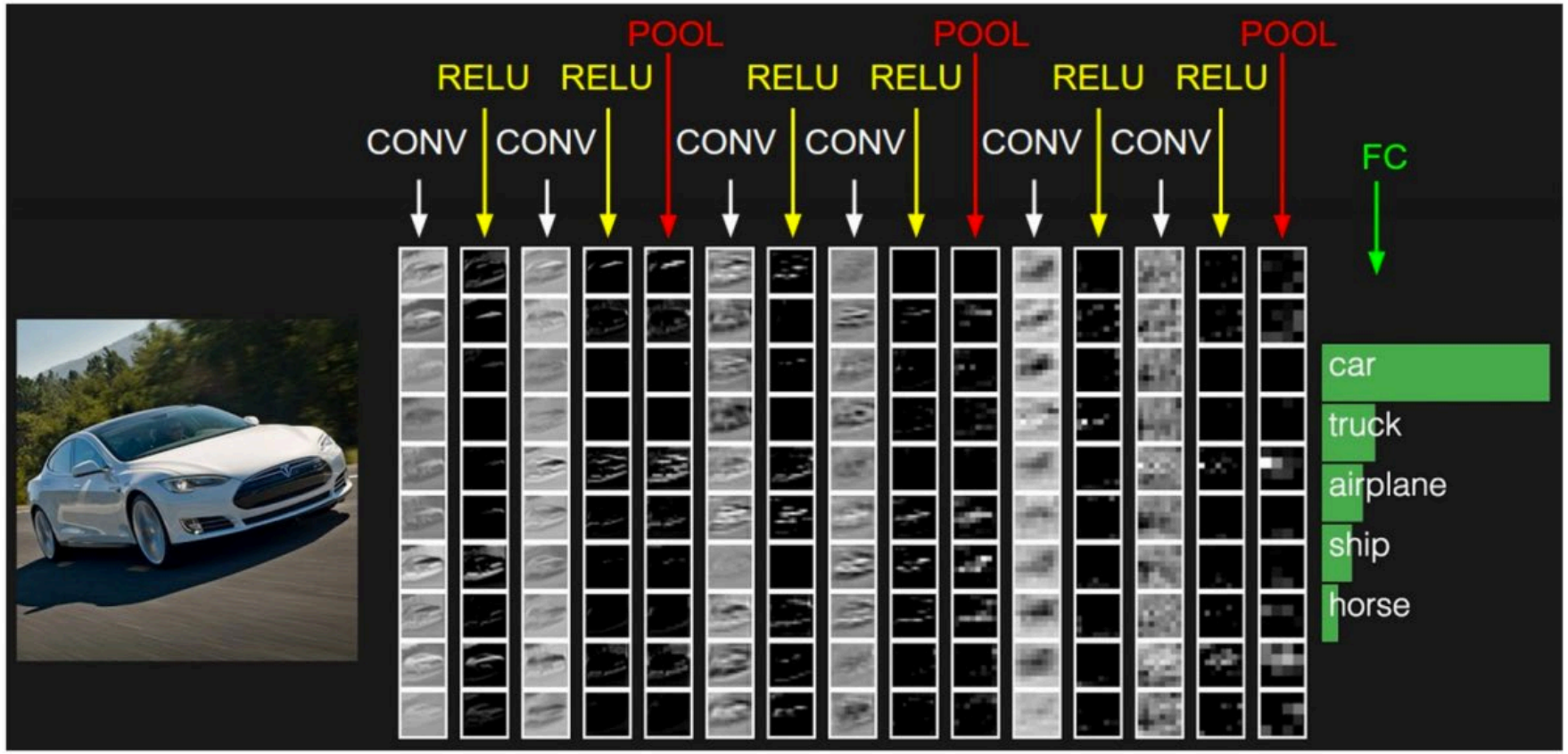
Demo







Repeated Convolutions & Pooling

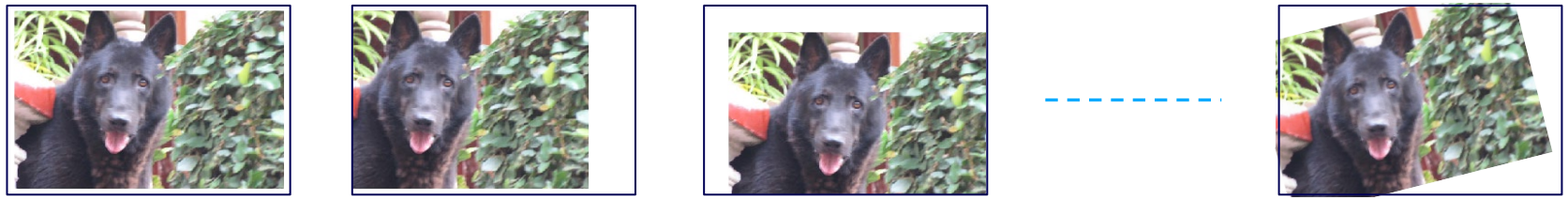


RMIT Classification: Trusted

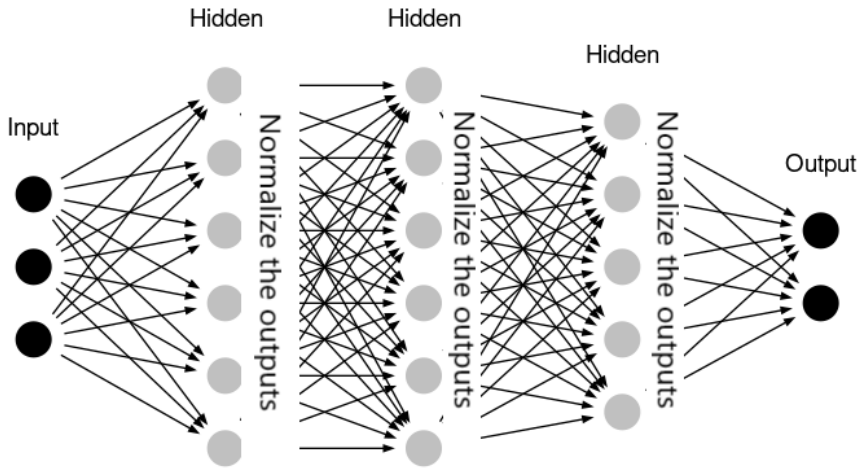
Practical Issues

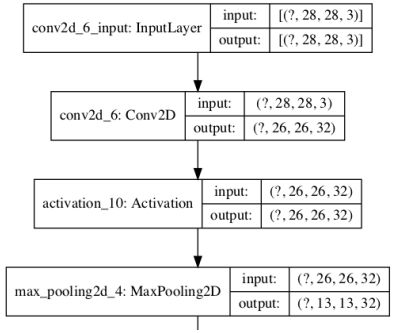
Overfitting

- Regularization
- Dropout
- Early Stopping
- Data Augmentation: Images can be augmented with random image manipulation operations such as translations, rotations, zooming etc.

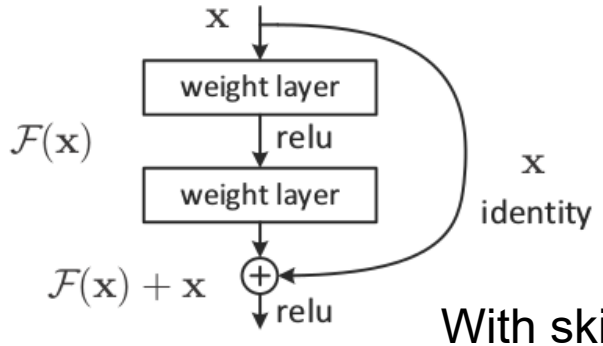


Vanishing Gradients





No skip connections



With skip connections

Popular CNN Architectures (Classification)

Convolutional Neural Networks have been around since early 1990s. Some influential architectures are:

- LeNet (1990s)

  One of the very first CNN convolutional neural networks.

- AlexNet (2012)

  A deeper and much wider version of the LeNet

  Won, by a large margin, the difficult ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012.

  It was a significant breakthrough with respect to the previous approaches and the current CNNs approaches can be attributed to this work.

- ZF Net (2013)

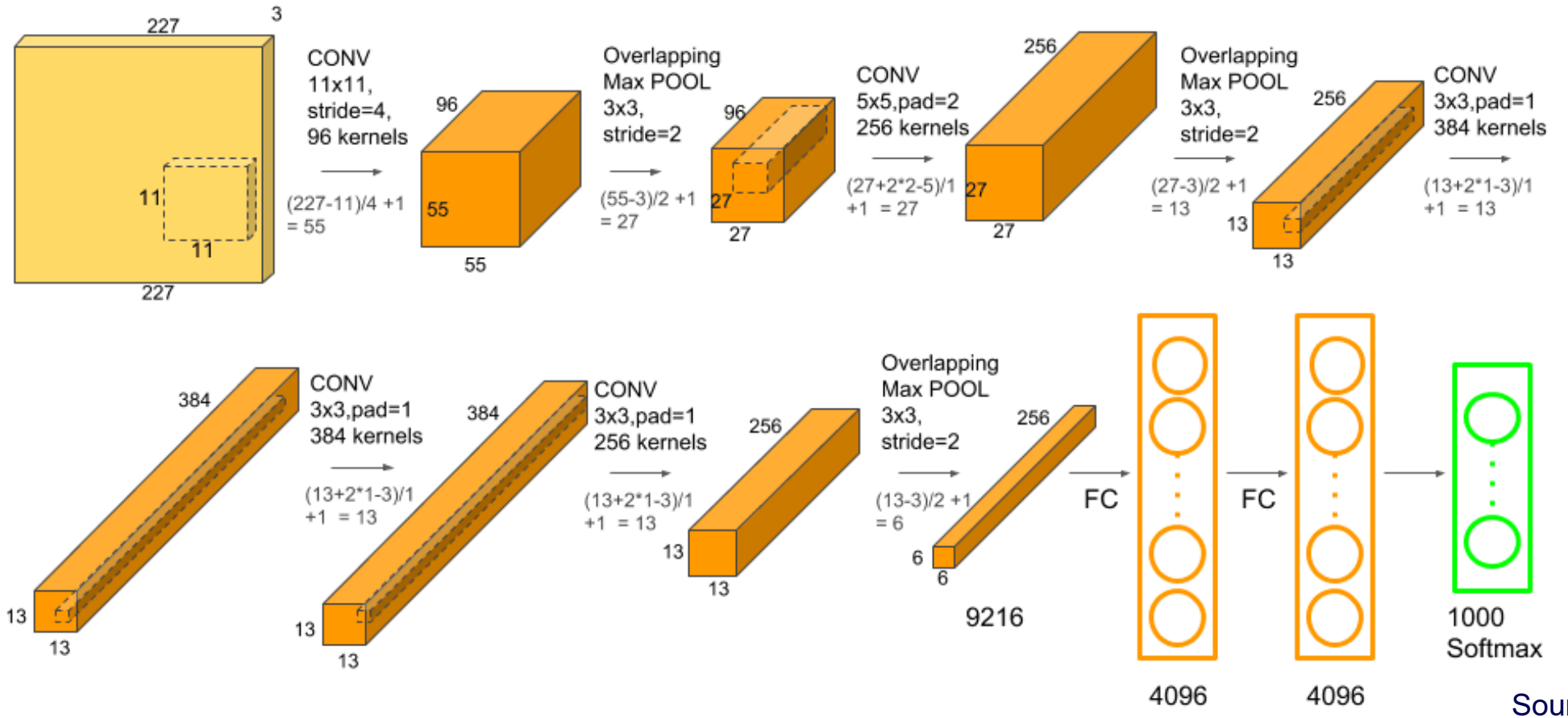
  The ILSVRC 2013 winner

  An improvement on AlexNet by tweaking the architecture hyper-parameters.

Popular CNN Architectures (Classification)

- GoogLeNet (2014)
  - ILSVRC 2014 winner
  - Developed the Inception Module that dramatically reduced the number of parameters in the network (4M, compared to AlexNet with 60M).
- VGGNet (2014)
  - ILSVRC 2014 Runner-up
  - Showed that the depth of the network (number of layers) is a critical component for good performance.
- Others include ResNets (2015) & DenseNet (2016)
  - And many, many more since then

AlexNet (2012)



Source: neurohive.io

Model development

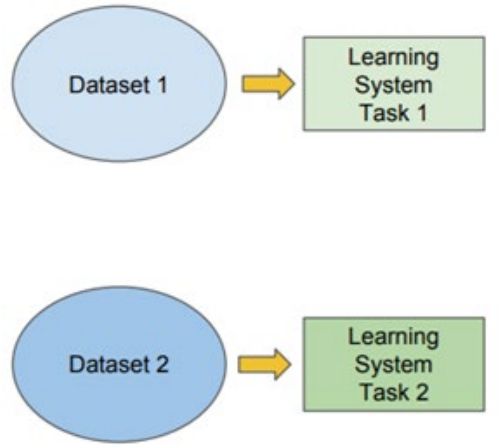
As discussed in Week 4-5, the typical ML model development process consists of 4 steps, lets go through each and see how it is done.

- **Determine your goals**: Performance metric and target value. Problem dependent.
- **Setup the experiment**: Setup the test/validation data, visualizers and debuggers needed to determine bottlenecks in performance (overfitting/under-fitting, feature importance).
- **Default Baseline Model**: Identify the components of end-to-end pipeline including - Baseline Models, cost functions, optimization.
- **Make incremental changes**: Repeatedly make incremental changes such as gathering new data, adjusting hyper-parameters, or changing algorithms, based on specific findings from your instrumentation.

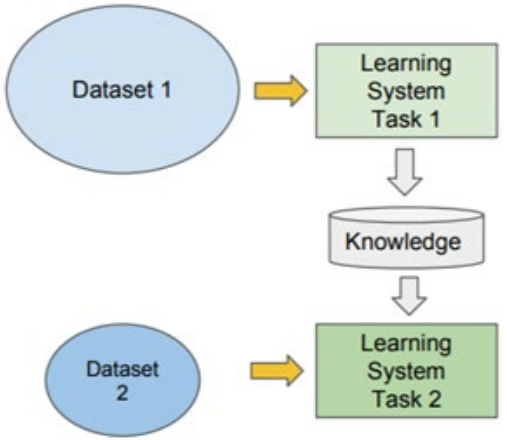
Transfer Learning

- Transfer learning is a machine learning method where a model developed for a task is reused as the starting point for a model on a second task.

  - **Select Source Model**. A pre-trained source model is chosen from available models. Many research institutions release models on large and challenging datasets that may be included in the pool of candidate models from which to choose from.
  - **Reuse Model**. The pre-trained model can then be used as the starting point for a model on the second task of interest. This may involve using all or parts of the model, depending on the modelling technique used.
  - **Tune Model**. Optionally, the model may need to be adapted or refined on the input-output pair data available for the task of interest.



Traditional Model development



Transfer learning

Observations

Neural Networks “work” as they are a generalisation of standard computer vision

- The network design is not “random” and not just fully connected networks
- While network design is sometimes seen as “black-magic” the most successful network designs are based on previously established approaches, replacing parameters, kernels, projections, etc. with perceptron-based units

Some important things we did not cover

- CNN is not limited to image classification. It is used in many other computer vision tasks including image segmentation, tracking, object detection, image captioning etc.
  - The preferred architectures change depending on the task.
- The CNN networks are also used to model speech and other types of tasks.
- Time series information is modeled using recurrent neural networks (RNN) and LSTM.
- Unsupervised learning
  - Autoencoders and Generative Adversarial Networks (GANs)

Readings and Acknowledgement

The following website provides a good explanation on CNN:
https://ujjwalkarn.me/2016/08/11/intuitive-explanation-convnets/

Andrew Ng's course on CNN at coursera:
https://www.coursera.org/learn/convolutional-neural-networks

A few slides were from the following source:
https://www.slideshare.net/LuMa921/deep-learning-a-visual-introduction

Further readings:

https://en.wikipedia.org/wiki/Convolutional_neural_network

Ian Goodfellow's DL book, chapter 9 on CNN: https://www.deeplearningbook.org/

12 Frequently Asked Questions on Deep Learning:
https://www.analyticsvidhya.com/blog/2018/05/deep-learning-faq/

RMIT Classification: Trusted

Assignment-2 Q&A

1. For Image analysis Project:

    * Baseline NN, Improvement can use DL
    * One folder for each task
    * Going through announcements one more time
    * Any Question?

2. For RL project:

    * The priority for each queue is given by the Mean Delay Requirement column in table 1, so you can see that 'Priority Queue 2' is higher priority (has a shorter mean delay requirement) than 'Priority Queue 1'.

RMIT Classification: Trusted

Assignment-2 Q&A

* The capacity of queues should be limited to a pre-defined threshold.
* You select a queue and transmit one packet within a timeslot
* The target can be: reduce delay in the best effort queue while remaining within the delay requirements of priority queues 1 and 2
* Going through announcements one more time
* Any Question?

Assignment-1

* Sample Solution
* Common Mistakes

Summary

Convolutional Neural Networks (CNNs)

- Convolutions
- ReLU
- Pooling
- MLP
- Strengths & Weaknesses
- Example CNNs